



Lecture 1

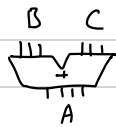
Motivation: Hardware $\xrightarrow{\text{faster}}$ Software

Abstraction of hardware computations

focus

Computation $A = B + C$

Adder



Logic Gates



Transistors



Silicon



Practical work

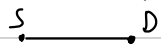
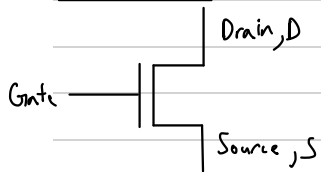
Hierarchy: Divide system into modules, further subdivide modules until pieces are easy to understand

Modularity: Modules have well-defined functions and interfaces st. they connect easily w/o side effects

Regularity: Uniformity among modules. Reusing common modules so that distinct modules don't need to be designed.

Transistors

When Gate Voltage == High:

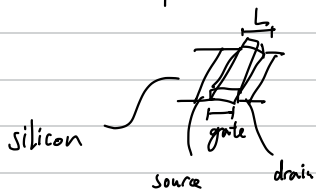


else:



Transistors = switches

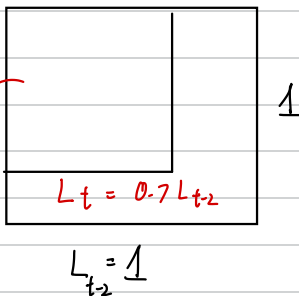
Silicon chips



- In 2025, $L \sim 10\text{nm}, 7\text{nm}$

Moore's law

- Every 18-24 months, transistor count $\times 2$
- Every 2 years, $L_{t+2} = 0.7 L_{t-2}$



$$A_t = 0.49 A_{t-2}$$

Hardware vs Software:

- Faster
- Can be stacked (Use 10 adders instead of 4)

- Slower
- General

- Hardwares (such as transistors, gates, 8-bit adders still take time)

2-3ps 40ps 300ps

- Travelling b/w wires also need time
 - ↳ Resistance
 - ↳ Capacitance $\rightarrow$ RC time const.

Lecture 2

Number systems

Decimal (base 10)

- count your toes
- ↳ feet

Binary (base 2)

- Digits: 0, 1
- Format: 0b100 = 4₁₀

Hexadecimal (base 16)

- Digits: 1-9, A-F
- Format: 0xA = 26₁₀

Misc

- Babylonian (Clock / Angle) $\rightarrow$ base 60
- Mayans $\rightarrow$ base 20

More on binary:

- You can represent 2^n values with n digits (bits)
- Binary to hexadecimal:

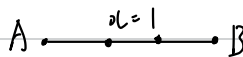
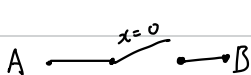
ex. 0b 1000 1100 1000
8 C 8 $\rightarrow$ 0x8C8

↳ * Since $2^4 = 16$

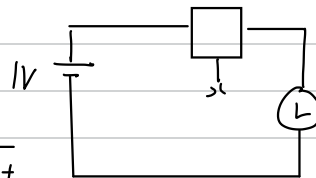
- Negative numbers $\rightarrow$ 2's complement
- Fraction (not relevant here)

Lecture 3

Transistors as switches



- Circuit:



- Logic expression:

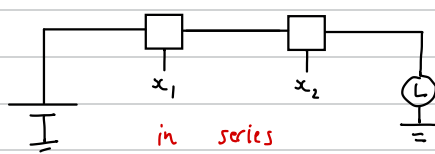
$$L = x, x \in \{0, 1\}$$

AND Gate

- Truth table:

x_1	x_2	L
0	0	0
0	1	0
1	0	0
1	1	1

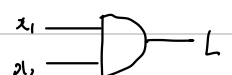
- Circuit:



- Logic Function:

$$L = x_1 x_2 = x_1 \cdot x_2$$

- Gate:

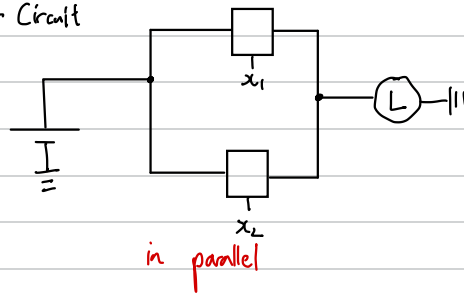


OR Gate

- Truth Table:

x_1	x_2	L
0	0	0
0	1	1
1	0	1
1	1	1

- Circuit



- Logic Function:

$$L = x_1 + x_2 = x_1 \vee x_2$$

- Gate

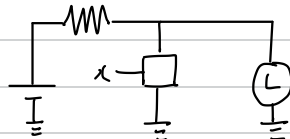


NOT Gate

- Truth Table:

x	L
0	1
1	0

- Circuit



- Logic Function:

$$\begin{aligned} L &= \bar{x} \\ &= x' \\ &= \sim x \\ &= !x \end{aligned}$$

- Gate



XOR Gate (Exclusive Or)

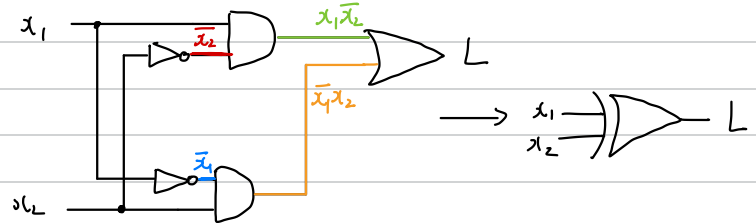
- Truth Table:

x_1	x_2	L
0	0	0
0	1	1
1	0	1
1	1	0

- Logic Function:

$$\begin{aligned} L &= \bar{x}_1 x_2 + x_1 \bar{x}_2 \\ &= x_1 \oplus x_2 \end{aligned}$$

- Gate



NAND Gate (Not And)

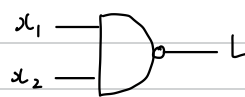
- Truth Table

x_1	x_2	L
0	0	1
0	1	1
1	0	1
1	1	0

- Logic Function:

$$\begin{aligned} L &= \bar{x}_1 \bar{x}_2 + \bar{x}_1 x_2 + x_1 \bar{x}_2 \\ &= \overline{(x_1 x_2)} \end{aligned}$$

- Gate



NOR Gate (Not Or)

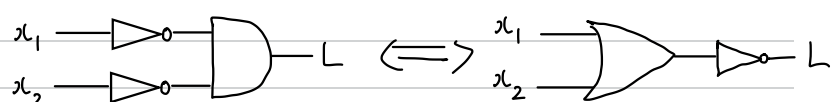
- Truth Table

x_1	x_2	L
0	0	1
0	1	0
1	0	0
1	1	0

- Logic Function:

$$\begin{aligned} L &= \overline{\bar{x}_1 \bar{x}_2} \\ &= x_1 + x_2 \end{aligned}$$

- Gate



Side Note: XNOR (Not XOR)

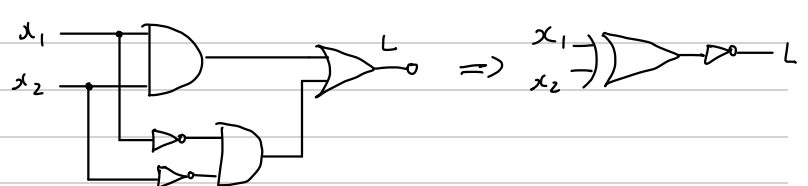
- Truth Table

x_1	x_2	L
0	0	1
0	1	0
1	0	0
1	1	1

- Logic Function:

$$\begin{aligned} L &= \bar{x}_1 \bar{x}_2 + x_1 x_2 \\ &= x_1 \odot x_2 \end{aligned}$$

- Gate



3-input XOR (or more)

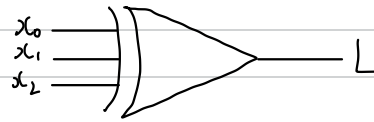
- Truth table

x_2	x_1	x_0	$x_0 \oplus x_1$	L
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

- Logic function

$$L = x_0 \oplus x_1 \oplus x_2$$

- Gate:



- For any XOR Gates, with n inputs, $n \geq 2$
the output is high for odd # of high inputs,
low otherwise
 $\hookrightarrow \begin{cases} L = 1, & \text{odd number of input 1} \\ L = 0, & \text{even number of input 1} \end{cases}$

Lecture 4

Canonical SOP

Sum of Products (SOP): an OR of AND terms

Minterm: A product term that evaluates to 1 for exactly 1 row of the truth table

Canonical SOP: SOP of Minterms

Example:

x_1	x_2	x_3	minterm	f
0	0	0	$\bar{x}_1 \bar{x}_2 \bar{x}_3$	1
0	0	1	$\bar{x}_1 \bar{x}_2 x_3$	
0	1	0	$\bar{x}_1 x_2 \bar{x}_3$	
0	1	1	$\bar{x}_1 x_2 x_3$	1
1	0	0	$x_1 \bar{x}_2 \bar{x}_3$	
1	0	1	$x_1 \bar{x}_2 x_3$	
1	1	0	$x_1 x_2 \bar{x}_3$	1
1	1	1	$x_1 x_2 x_3$	

- Then Canonical SOP, $f = \bar{x}_1 \bar{x}_2 \bar{x}_3 + \bar{x}_1 x_2 x_3 + x_1 x_2 \bar{x}_3$

Canonical POS

Product of Sums (POS): an AND of OR terms

Maxterm: A sum term that evaluates to 0 for exactly 1 row of the truth table

Canonical POS: POS of Maxterms

Example:

x_1	x_2	x_3	maxterm	f
0	0	0	$x_1 + x_2 + x_3$	
0	0	1	$x_1 + x_2 + \bar{x}_3$	0
0	1	0	$x_1 + \bar{x}_2 + x_3$	
0	1	1	$x_1 + \bar{x}_2 + \bar{x}_3$	0
1	0	0	$\bar{x}_1 + x_2 + x_3$	0
1	0	1	$\bar{x}_1 + x_2 + \bar{x}_3$	
1	1	0	$\bar{x}_1 + \bar{x}_2 + x_3$	0
1	1	1	$\bar{x}_1 + \bar{x}_2 + \bar{x}_3$	

- Canonical POS, $f = (x_1 + x_2 + \bar{x}_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + x_2 + x_3)(\bar{x}_1 + \bar{x}_2 + x_3)$

★ Summary

Canonical SOP	Canonical POS
1. Identify rows with 1	1. Identify rows with 0
2. Write out minterms $\bar{x}$ if 0, x if 1 ex. 001 $\rightarrow \bar{x}_2 \bar{x}_1 x_0$	2. Write out maxterms $\bar{x}$ if 1, x if 0 ex. 001 $\rightarrow x_2 + x_1 + \bar{x}_0$
3. OR them up ex. $x_2 \bar{x}_1 + x_2 x_1$	3. AND them up ex. $(x_2 + \bar{x}_1)(\bar{x}_1 + \bar{x}_1)$

- They should be equal.

Lecture 5

Boolean Algebra

Duality: An expression is valid if $0 \Leftrightarrow 1$ • $\Leftrightarrow +$

Axioms

Base Form	Dual Form
1. $0 \cdot 0 = 0$	$1 + 1 = 1$ AND/OR
2. $1 \cdot 1 = 1$	$0 + 0 = 0$ AND/OR
3. $0 \cdot 1 = 1 \cdot 0 = 0$	$1 + 0 = 0 + 1 = 1$ AND/OR
4. If $x=0$, then $\bar{x}=1$	If $x=1$, then $\bar{x}=0$ Not
0? $x=0$, if $x \neq 1$	$x=1$, if $x \neq 0$ Binary field

Rules

Base Form	Dual Form
5. $x \cdot 0 = 0$	$x + 1 = 1$ Null Element
6. $x \cdot 1 = x$	$x + 0 = x$ Identity
7. $x \cdot x = x$	$x + x = x$ Idempotency
8. $x \cdot \bar{x} = 0$	$x + \bar{x} = 1$ Complements
9. $\bar{\bar{x}} = x$	— None — Involution

Identities

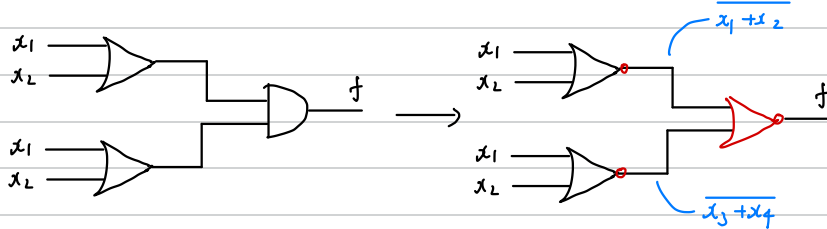
Base Form	Dual Form
10. $x \cdot y = y \cdot x$	$x + y = y + x$ Commutativity
11. $x \cdot (y \cdot z) = (x \cdot y) \cdot z$	$x + (y + z) = (x + y) + z$ Associative
12. $x \cdot (y + z) = xy + xz$	$x + (yz) = (x + y)(x + z)$ Distributivity
13. $x + x \cdot y = x$	$x \cdot (x + y) = x$ Absorption / Covering
14. $xy + x\bar{y} = x$	$(x + y)(x + \bar{y}) = x$ Combining
15. $\overline{x_1 \cdot x_2 \cdot x_3 \dots} = \bar{x}_1 + \bar{x}_2 + \bar{x}_3 \dots$	$\overline{(x_1 + x_2 + x_3 + \dots)} = \bar{x}_1 \cdot \bar{x}_2 \cdot \bar{x}_3 \dots$ De Morgan's Theorem
16? $(x \cdot y) + (\bar{x} \cdot z) + (y \cdot z)$ $= (x \cdot y) + (\bar{x} \cdot z)$	$(x + y) \cdot (\bar{x} + z) \cdot (y + z)$ $= (x + y) \cdot (\bar{x} + z)$ Consensus Redundancy term Complemented variable
17 $x + \bar{x} \cdot y = x + y$	$x(\bar{x} + y) = x \cdot y$



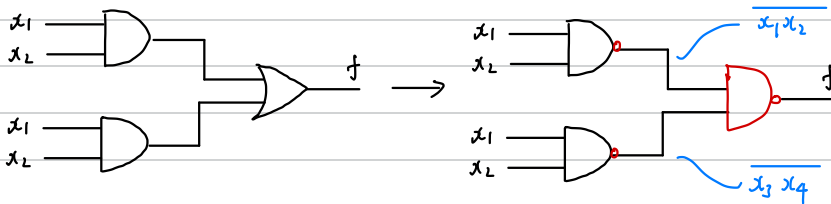
★ More on DeMorgan's Theorem

- Any SOP circuit can be implemented as NAND-NAND → Save transistors → AND/OR: 6 transistors
- Any POS circuit can be implemented as NOR-NOR → NAND/NOR: 4 transistors

- POS, ex. $f = (x_1 + x_2)(x_3 + x_4) = \overline{\overline{(x_1 + x_2)} \overline{(x_3 + x_4)}} = \overline{(\overline{x_1} \overline{x_2}) (\overline{x_3} \overline{x_4})}$ (base form DeMorgan's)



- SOP, ex. $f = x_1 x_2 + x_3 x_4 = \overline{\overline{x_1 x_2} \overline{x_3 x_4}} = \overline{(\overline{x_1} \overline{x_2}) (\overline{x_3} \overline{x_4})}$ (dual form DeMorgan's)



Lecture 7

Karnaugh Map

- Used for logic optimization
- Circle adjacent cells of size 2^n , wrap around allowed.
- Use Gray code (00 01 11 10), adjacent entries differ by 1 variable

- Terminology

↳ Implicants: Any product term covered by f
ex. $m_0, m_1, m_2, \dots, \bar{x}\bar{y}, \bar{z}$ etc.

↳ Prime Implicants: an implicant for which it is not possible to remove any literal and still have a valid implicant

ex. $x\bar{z} \rightarrow \bar{z}$ ✓, $x\bar{z}$ is not a prime implicant

ex. $\bar{x}\bar{y} \rightarrow \bar{x}$ or $\bar{y}$, not defined in example, $\bar{x}\bar{y}$ is a prime implicant

↳ Cover: any set of implicants that includes all minterms of a function

ex. $\bar{z}, \bar{x}\bar{y}\bar{z}$ or $\bar{z}, \bar{x}\bar{y}$

Example:

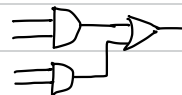
xy \ z	00	01	11	10
0	0	0	1	1
1	1			

Gray code

wrap around allowed

★ - Cost $\approx (\# \text{ of gates}) + (\# \text{ gate inputs})$
ex. $C = 2\text{AND} + 1\text{OR} + 2 \times 2 + 1 \times 2 = 9$ (unitless)

Example:



- Don't Cares

↳ Denoted as d , treated as either 0 or 1

↳ Choose accordingly to minimize cost

★ Summary



Step 1: Find PIs (circle)

Step 2: Identify essential PIs and include in the cover

Step 3: Choose the suitable PIs, so that
1. all minterms are covered
2. cost is minimized (minimizing literals)

- Notes:

↳ Circle "overlappingly" to find all POS

↳ We can circle 0s (maxterms) to create a POS instead.

Example

x	y	z	f
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	d
1	0	1	1
1	1	0	0
1	1	1	0

xy \ z	00	01	11	10
0	1	0	0	d
1	1	1	0	1

xy \ z	00	01	11	10
0	1	0	0	d
1	1	1	0	1

here d is treated as 1

↳ Then $f = \bar{y} + \bar{x}z$

Lecture 6a.

What is System Verilog

- Hardware Description Language
- CAD optimizes the circuit.

Combinational Logic

- Only uses current input values to compute output
- No storage

Basic Commands

```
module module_name (input logic a, b, // inputs
                    output logic w, x, y, z; // outputs
    assign w = a & b; AND
    assign x = a | b; OR
    assign y = ~a; NOT
    assign z = a ^ b; XOR
endmodule
```

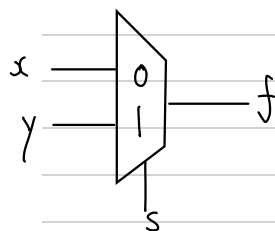
- assign: for cont. assignment
- logic: type, signals are Boolean variables

Multiplexer (mux)

- Selects x for $s=0$, else y

(2 to 1)

Hardware block



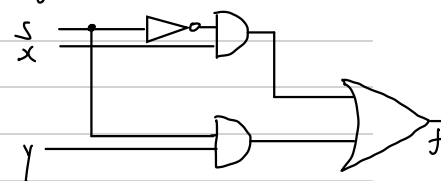
Truth Table

s	x	y	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Boolean expression

$$f = \bar{s}x\bar{y} + \bar{s}xy + s\bar{x}y + sy$$
$$= \bar{s}x + sy$$

Logic Gate



Verilog code

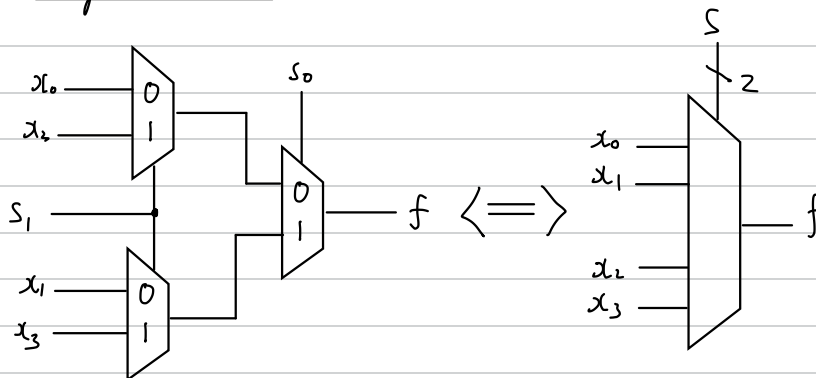
```
module mux 2to1 (input logic x,y,s,  
                  output logic f);  
    assign f = (~s & x) | (s & y);  
end module
```

(4 to 1)

Boolean expression

$$f = \bar{s}_1\bar{s}_0x_0 + \bar{s}_1s_0x_1 + s_1\bar{s}_0x_2 + s_1s_0x_3$$
$$= s_0(\bar{s}_1x_1 + s_1x_3) + \bar{s}_0(\bar{s}_1x_0 + s_1x_2)$$

Using mux 2to1

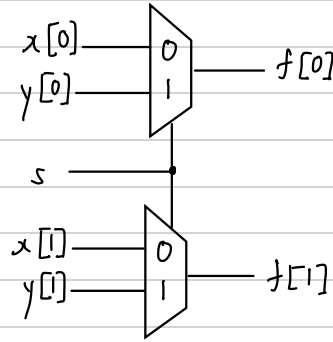
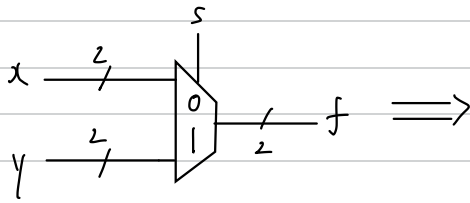


Verilog code

```
module mux 4to1 (input logic x-0,x-1,x-2,x-3,  
                 input logic [1:0] s,  
                 output logic f);  
    always-comb  
    begin  
        case (logic)  
            2'b 0: f = x-0;  
            2'b 1: f = x-1;  
            2'b 2: f = x-2;  
            2'b 3: f = x-3;  
            default: f = 0;  
        end case  
    end  
end module
```

(Multibit max)

Hardware block



Verilog

```
module mux_2to1 (input logic [1:0] x, y,
                 input logic s,
                 output logic f);
    assign f[0] = (~s & x[0]) | (s & y[0]);
    assign f[1] = (~s & x[1]) | (s & y[1]);
endmodule
```

Half adder

↳ Only adds 1 bit value → $a + b$, + is addition, not OR

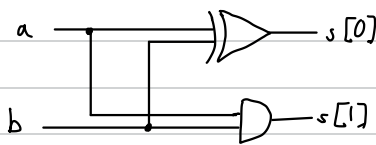
Truth table

Boolean expression

Logic Gates

a	b	s[1]	s[0]
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$s[0] = a \oplus b$
 $s[1] = ab$



System Verilog

```
module HA (input logic a, b,
           output logic [1:0] s);
    assign s[1] = a & b;
    assign s[0] = a ^ b;
endmodule
```

Full adder

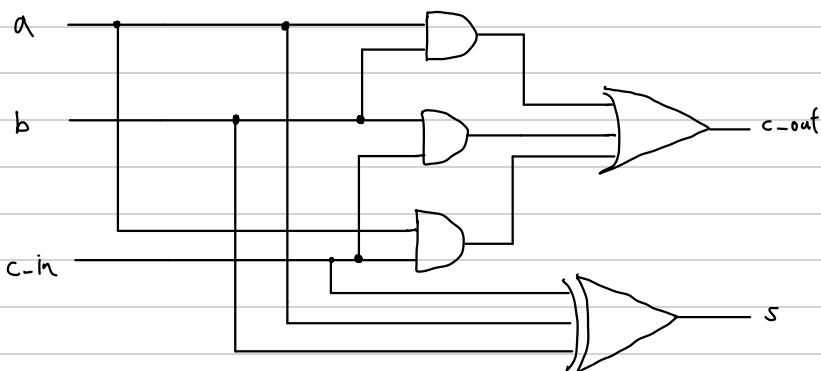
- ↳ Adds multibit numbers
- ↳ Carry out as output
- Carry in as input
- ↳ Combine multiple modules to form one

Truth table

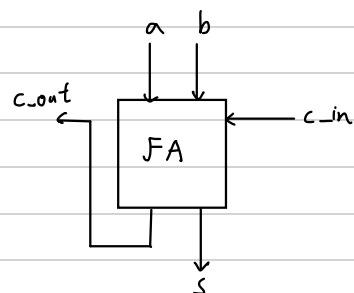
Boolean expression

a	b	c-in	s	c-out	
0	0	0	0	0	$s = \bar{a}\bar{b}c_{in} + \bar{a}b\bar{c}_{in} + a\bar{b}\bar{c}_{in} + abc_{in}$
0	0	1	1	0	$= \bar{a}(\bar{b}c_{in} + b\bar{c}_{in}) + a(\bar{b}\bar{c}_{in} + bc_{in})$
0	1	0	1	0	$= \bar{a}(b \oplus c_{in}) + a(\overline{b \oplus c_{in}})$
0	1	1	0	1	$= a \oplus b \oplus c_{in}$
1	0	0	1	0	$c_{out} = \bar{a}bc_{in} + a\bar{b}c_{in} + ab\bar{c}_{in} + abc_{in}$
1	0	1	0	1	$= \bar{a}bc_{in} + a\bar{b}c_{in} + ab\bar{c}_{in} + abc_{in} + ab-c_{in} + abc_{in}$
1	1	0	0	1	$= bc_{in}(\bar{a}+a) + a-c_{in}(b+b) + ab(c_{in}+c_{in})$
1	1	1	1	1	$= bc_{in} + a-c_{in} + ab$

Gates



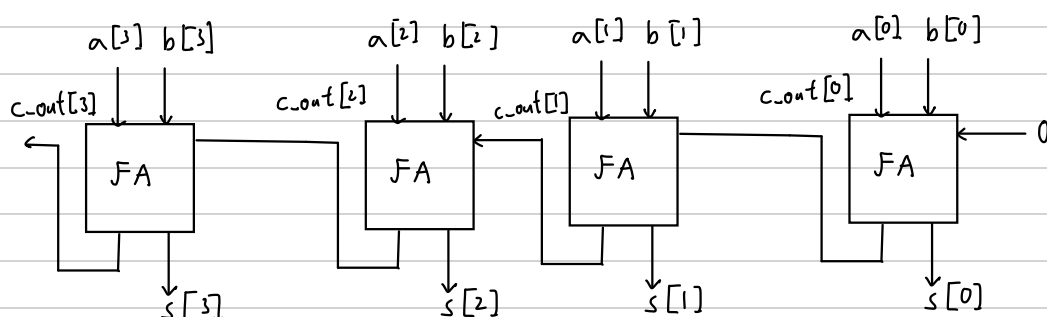
Block diagram



System Verilog

```
module FA(input a,b,c-in,  
          output c-out,s);  
    assign c-out = (a & b) + (a & c-in) + (b & c-in);  
    assign s = a ^ b ^ c-in;  
endmodule
```

Multibit FA (Lab 3 Part I)



```
module 4_bit_FA(input logic [3:0] a, b,
```

```
input logic c-in,
```

```
output logic [3:0] c-out, s); // here c-out is a vector, not necessarily required, can just output
```

Hierarchical FA FA0(a[0], b[0], c-in, c-out[0], s[0]); the last value

approach FA FA1(a[1], b[1], c-out[0], c-out[1], s[1]);

FA FA2(a[2], b[2], c-out[1], c-out[2], s[2]);

↓ FA FA3(a[3], b[3], c-out[2], c-out[3], s[3]);

use simpler submodule

modules

HEX display

Actual display

Truth table

	x_3	x_2	x_1	x_0	h_0	h_1	h_2	h_3	h_4	h_5	h_6
h_0	0	0	0	0	0	0	0	0	0	0	1
h_5	0	0	0	1	1	0	0	1	1	1	1
h_6	0	0	1	0	0	0	1	0	0	1	0
h_4	0	0	1	1	0	0	0	0	1	1	0
h_2	0	1	0	0	1	0	0	1	1	0	0
h_3	0	1	0	1	0	1	0	0	1	0	0
	0	1	1	0	0	1	0	0	0	0	0
	0	1	1	1	0	0	0	1	1	1	1
	1	0	0	0	0	0	0	0	0	0	0
	1	0	0	1	0	0	0	0	1	0	0

Beware!

- 0 when it is lit up

- 1 when it is off

System Verilog

```
module seg7(input logic [3:0] x,
```

```
output logic [6:0] h);
```

```
always-comb
```

```
begin
```

```
case(x)
```

```
4'b0000: h = 7'b0000001;
```

```
4'b0001: h = 7'b1001111;
```

```
⋮
```

```
4'b1001: h = 7'b0000100;
```

```
default: h = 7'b1111111;
```

```
end case
```

```
end
```

```
endmodule
```

Notes on System Verilog

- If ... else blocks must go inside always - comb

↳ always - comb

begin

if (...)

...

else if (...)

...

else

...

end

- case statements

↳ need default case

always - comb

begin

case (logic)

2'b00: x = ...;

2'b01: x = ...;

...

default: x = 1'b0;

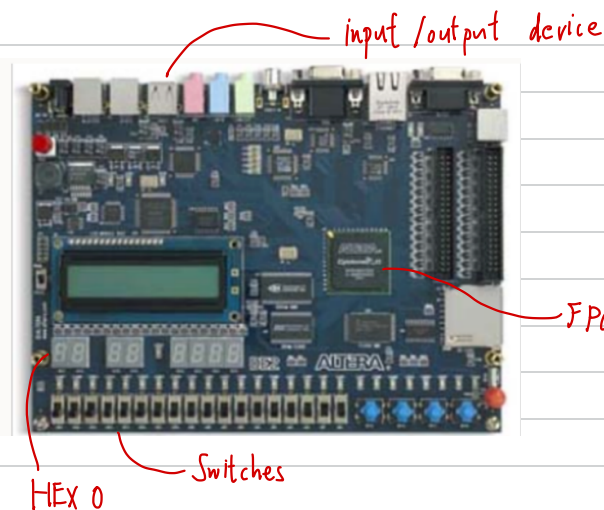
end case

end

↳ Generally, no instantiations can be done in if statements.

Lecture 6b

FPGA



- Field Programmable Gate Array

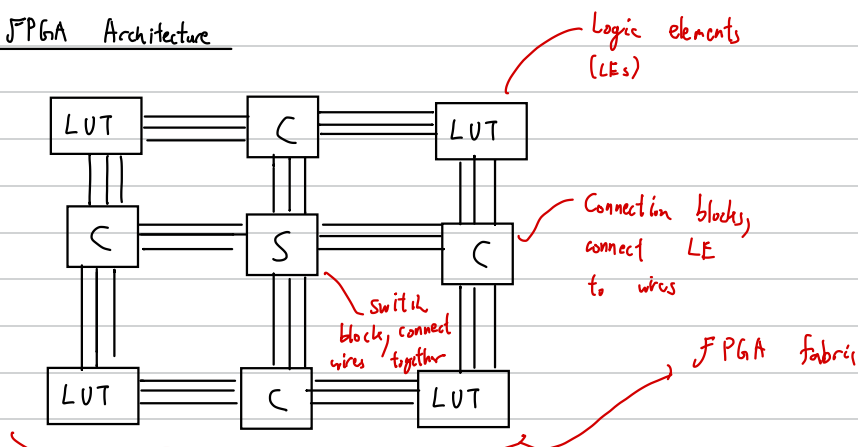
- Allow programmable wiring

↳ Different gates can connect to each other

- Makes gates programmable too

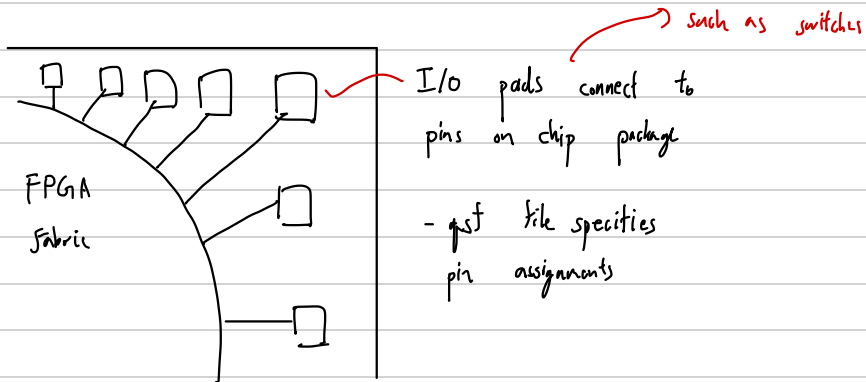
↳ Called a Look-Up-Table (LUT)

FPGA Architecture

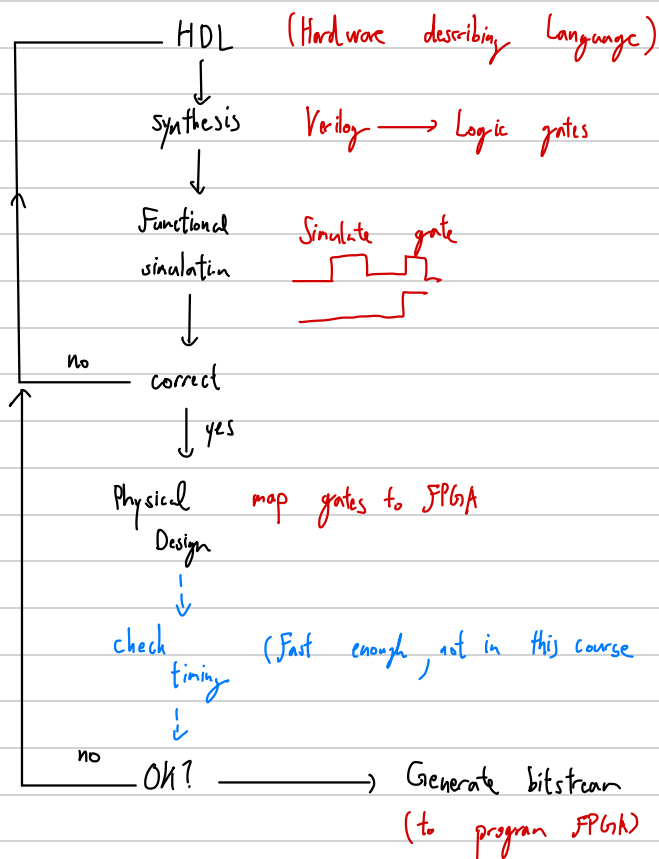


Configuration bits program the logic elements and switch connections

↳ Programming bitstreams



CAD flow

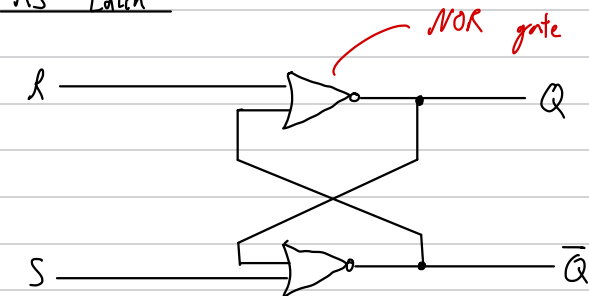


Lecture 8

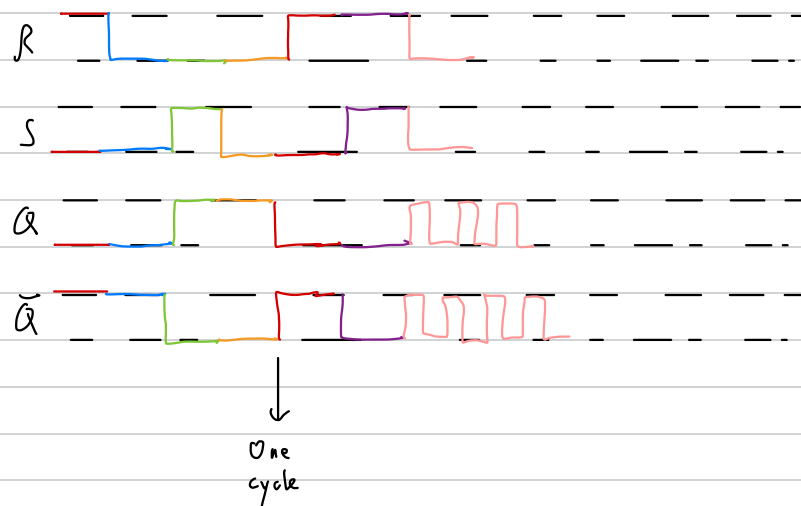
Sequential circuits

↳ Outputs determined by both present input and previous inputs

RS Latch



Timing diagram



Characteristic Table

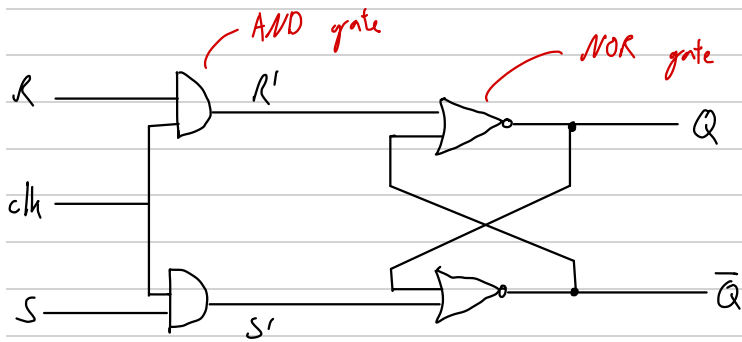
S	R	Q	Q̄	
0	0	Q ₀	Q̄ ₀	(Previous/stored state)
0	1	0	1	
1	0	1	0	
1	1	0	0	(not used)

↳ R (Reset), S (Set)

When R high → Q gets 0, Q̄ gets 1
S high → Q gets 1, Q̄ gets 0

Gated RS Latch

Refers to the AND gates

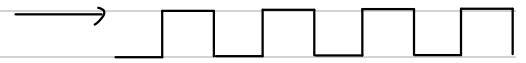


Characteristic Table

S	R	clk	$Q(t+1)$
X	X	0	$Q(t)$ ← Prev. state
0	0	1	$Q(t)$ ← Prev. state
1	0	1	1
0	1	1	0
1	1	1	Not used

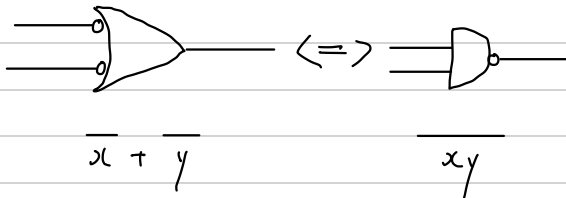
Clock

Square-wave generated by crystal oscillator



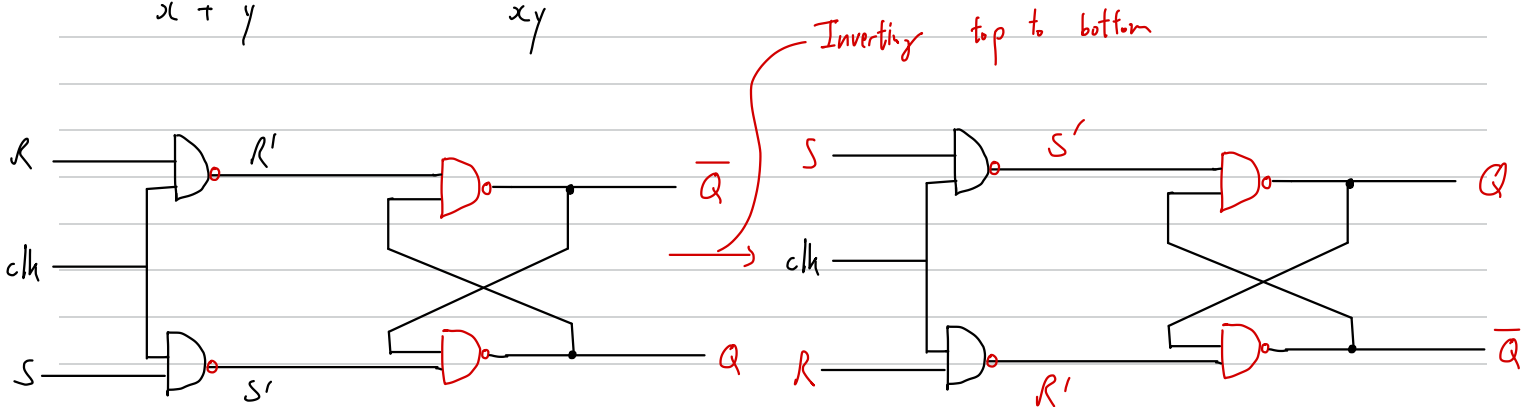
NAND-Gated RS latch

- Apply De Morgan's Theorem



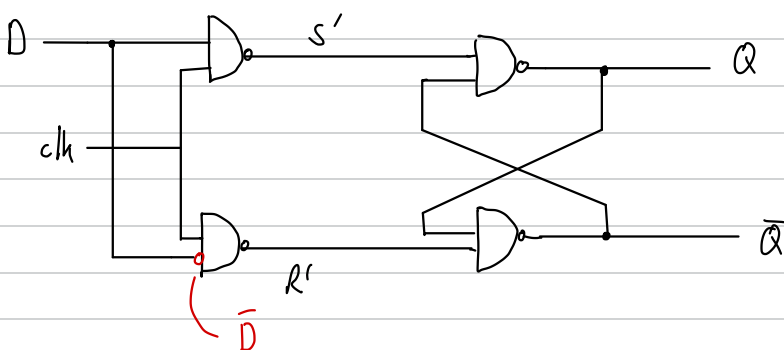
$$x + \bar{y}$$

$$\overline{xy}$$



Gated D latch

→ Let D be S, $\bar{D}$ be R



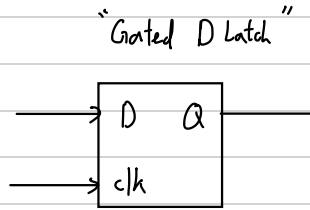
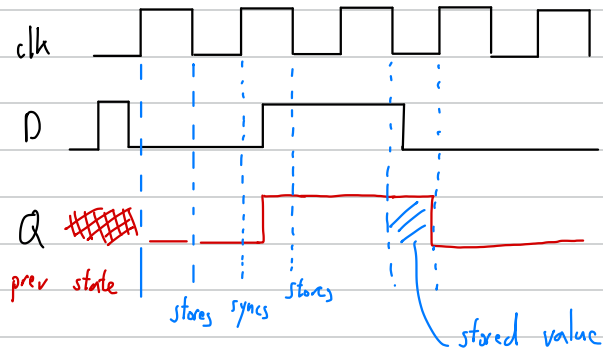
Characteristic table

D	clk	$Q(t+1)$
X	0	$Q(t)$ store
0	1	0 reset
1	1	1 set

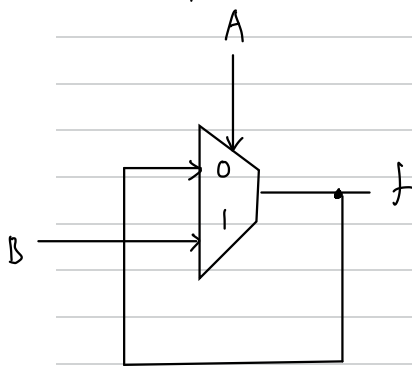
- D is the data input

- When $clk = 1$, $Q = D$

- When $clk \rightarrow 0$ (drops to 0), Q stores D "level sensitive"



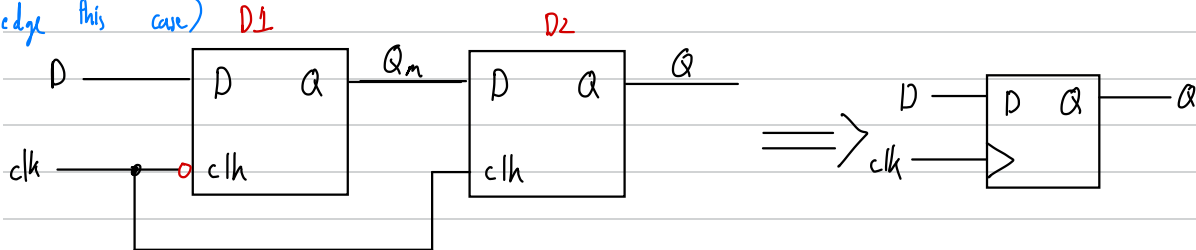
Max implementation



A	B	f(t+1)
0	X	f(t)
1	0	0
1	1	1

Flip Flops

(posedge this case) D1

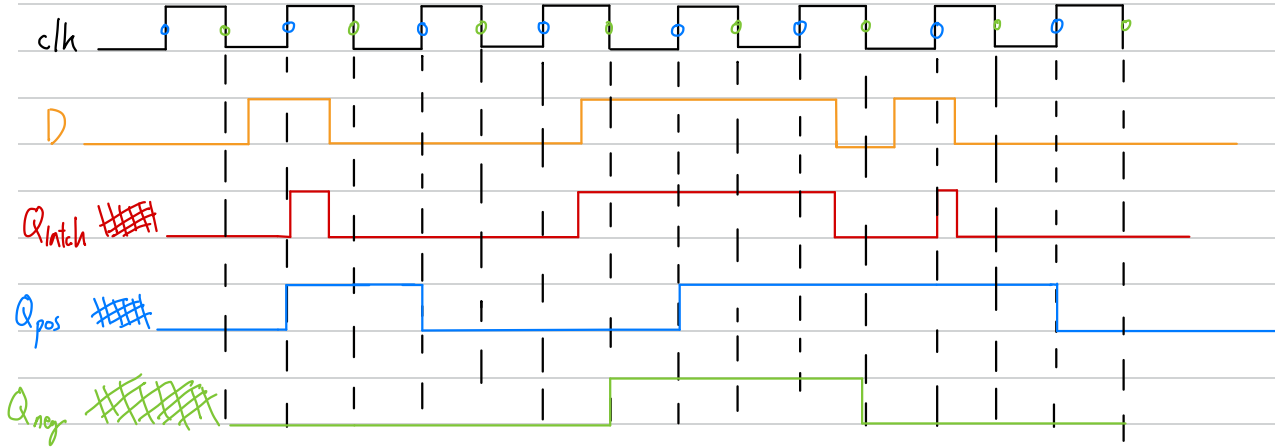
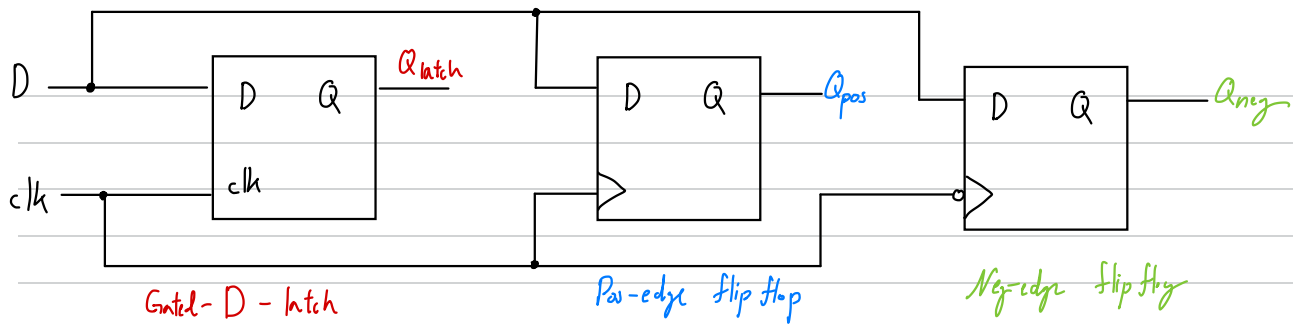


- When $clk = 0$: D1 : $Q_m = D$

D2 : Stores Q_m , $Q = \text{Prev state}$

- When $clk \uparrow$ (Rising / Pos edge) : $Q = Q_m = D$
(only at that moment)

- Also known as registers



Verilog for D latch

```

module D_latch (input logic D, clk,
                output logic Q);
    always_latch
    if (clk == 1)
        Q = D; // stores Q otherwise
endmodule

```

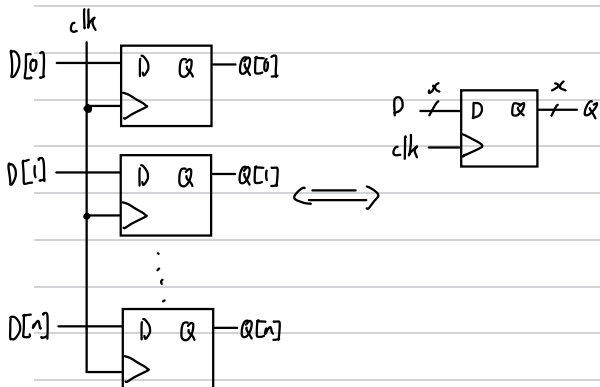
Verilog for FFs

```

module D_FF (input logic clk, D,
             output logic Q);
    always_ff @(posedge clk) // (neg-edge clk) if neg-edge triggered.
        Q <= D;
endmodule

```

Registers (multibit FFs)



Verilog for Registers

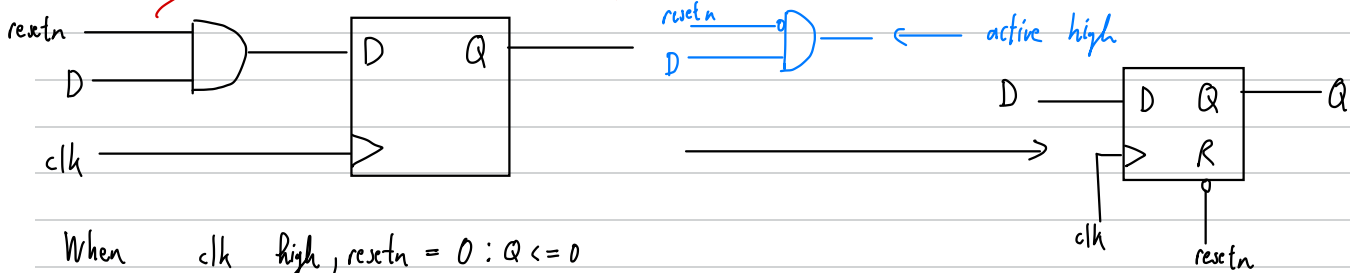
```

module reg4 (input logic clk,
             input logic [3:0] D,
             output logic [3:0] Q);
    always_ff @(posedge clk)
        Q <= D;
endmodule

```

Synchronous Resets

known as active low, i.e. resets when resetn is low

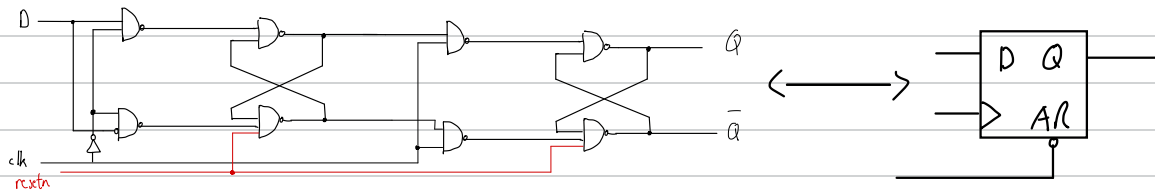


When $\text{clk} \text{ high}, \text{resetn} = 0 : Q \leftarrow 0$
 $\text{clk} \text{ high}, \text{resetn} = 1 : Q \leftarrow D$
 $\text{clk} \text{ low}, \text{resetn} = X : Q \text{ unchanged, in previous state}$

Verilog:

```
module D_FF (input logic clk, D, resetn
              output logic Q);
    always_ff @ (posedge clk) // synchronous inside posedge clk
        if (!resetn) Q <= 1'b0; // active low, resets when resetn == 0
        else Q <= D;
endmodule
```

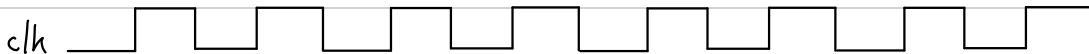
Asynchronous Reset

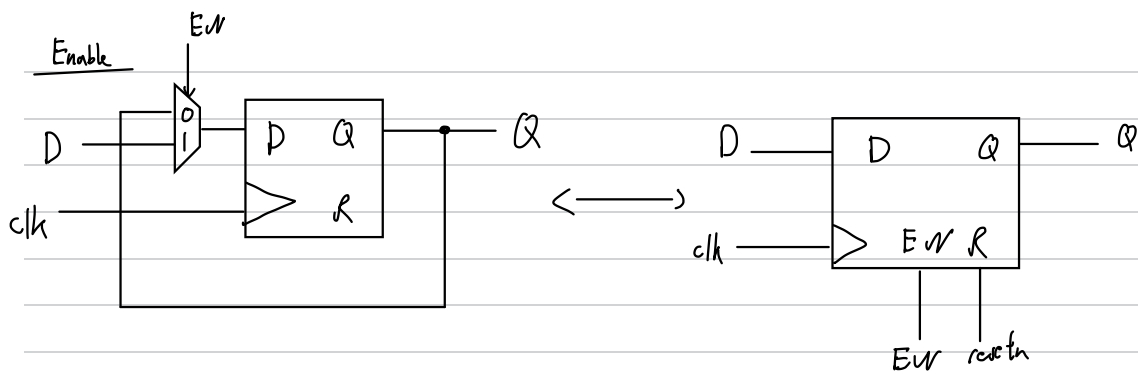


When $\text{clk} = 0, 1, \text{resetn} \rightarrow$ Gives $\bar{Q} = 1, Q = 0$ (clk does not matter)

Verilog:

```
module D_FF (input logic clk, D, resetn
              output logic Q);
    always_ff @ (posedge clk, negedge resetn) // resetn in sensitivity list, directly affect Q
        begin
            if (!resetn) Q <= 1'b0;
            else Q <= D;
        end
endmodule
```





- "Enables" change at clock edge, else reuses previous Q

Counters

Clock cycle	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

- Toggle is done by $Q = Q \oplus 1$

$$Q_0(t+1) = 1 \oplus Q_0(t)$$

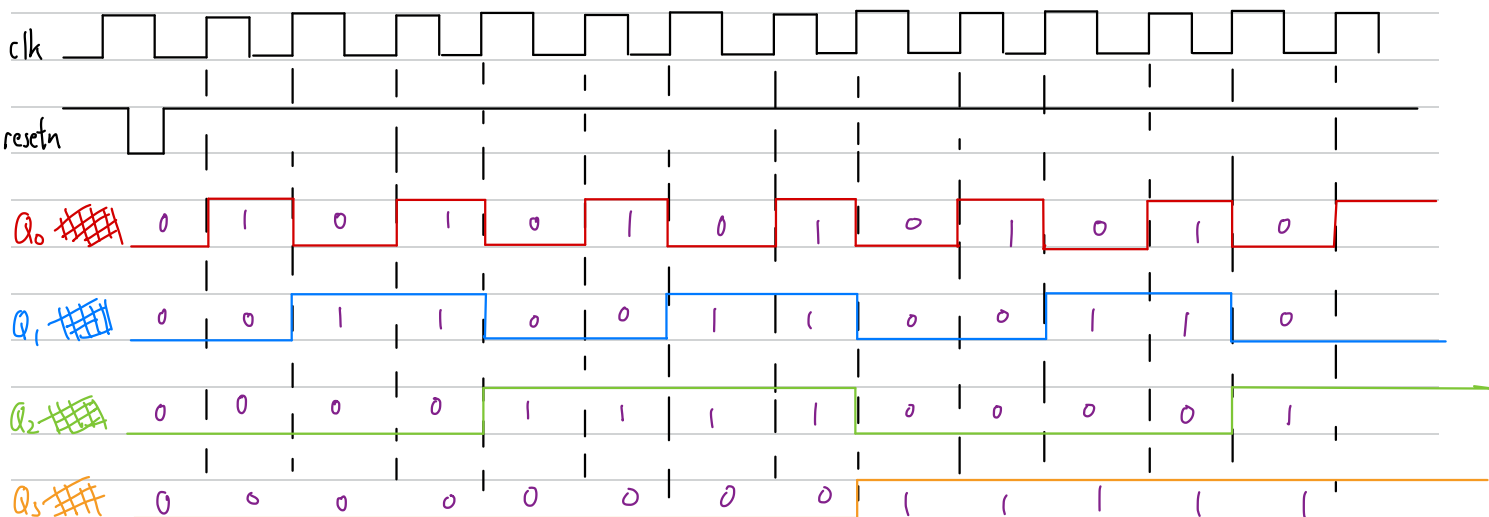
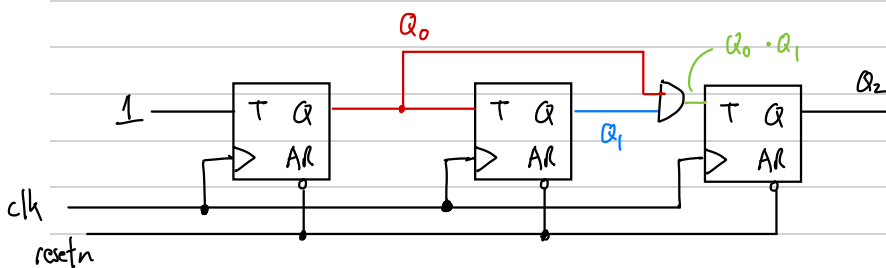
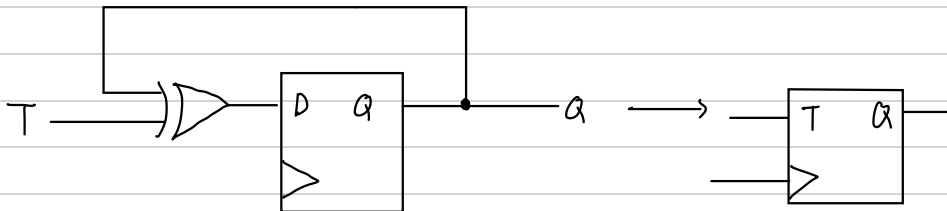
$$Q_1(t+1) = Q_1(t) \oplus Q_0(t), Q_1 \text{ toggles when previous } Q_0, Q_0(t)=1$$

$$Q_2(t+1) = Q_2(t) \oplus (Q_1(t) \cdot Q_0(t)), Q_2 \text{ toggles when previous } Q_0, Q_1 \text{ are both } 1 \rightarrow Q_0 Q_1 = 1$$

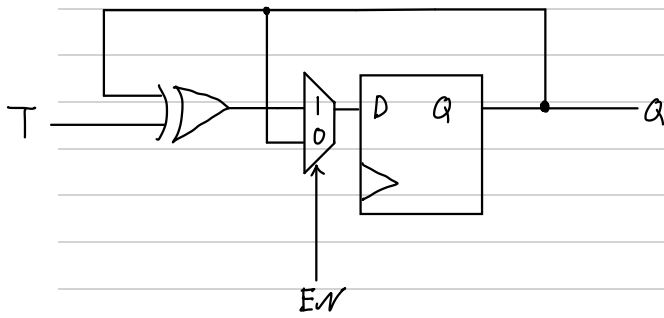
$$Q_3(t+1) = Q_3(t) \oplus (Q_2 Q_1 Q_0)$$

$\vdots$

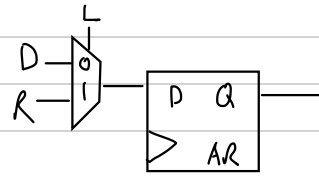
TQ Flip Flop



Pausing (Enable)



Parallel load (Load some numbers we want)



Verilog

Counter with **parallel load**, **enable**, **asynchronous reset**

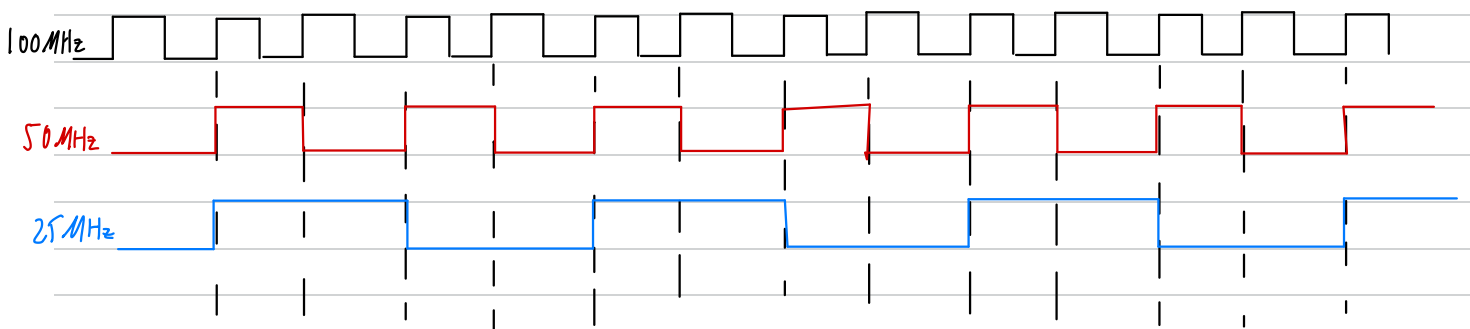
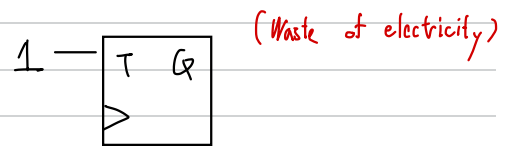
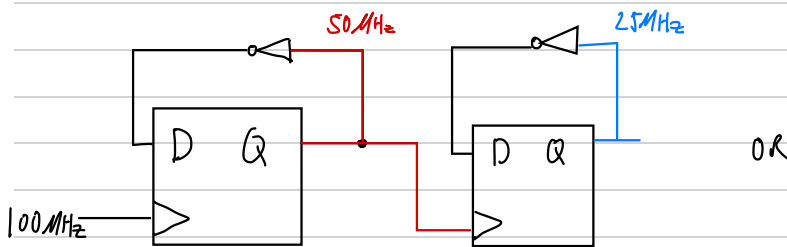
```

module upcount (input logic [3:0] R,
                 input logic rstn, clk, EN, L,
                 output logic [3:0] Q);
    always_ff @(posedge clk, negedge rstn)
    begin
        if (!rstn)
            Q <= 4'b0; // reset has highest priority
        elseif (L)
            Q <= R; // if parallel load high, load specific value
        else if (EN)
            Q <= Q + 1; // if enabled, count 1 more
    end
endmodule

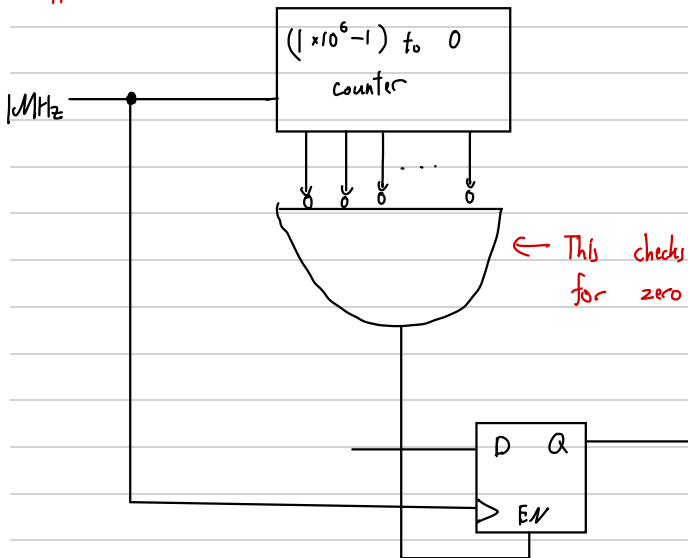
```

Rate Dividers

Approach 1, D flip flops



Approach 2: Counters (Lab 4)



— Counts down from million & start again after reaching 0

— To divide rate by 2 we count $\text{freq}/2 - 1$ to 0

↳ To divide rate by n we count $\text{freq}/n - 1$ to 0

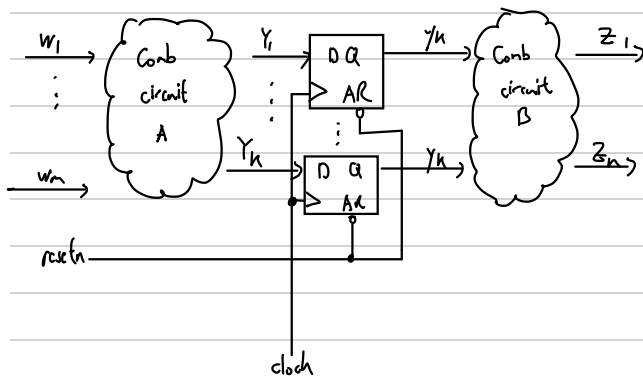
← This checks for zero

Lecture 9

Finite State Machines

↳ Any general sequential circuit

↳ Since a circuit with n registers can be in one of a finite number (2^n) unique states

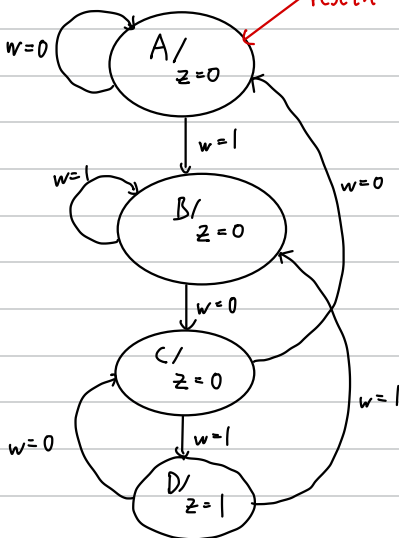


Steps to creating FSM

Example: if status is fine, $z=0$, status is not fine if a sequence of 1,0,1 in w occurs

↳ Step 1: State Diagram

resetn (typically enter first state w/ reset)



↳ Step 2 State Table

PS (Present state)	Next state		Output z
	$w=0$	$w=1$	
A	A	B	0
B	C	B	0
C	A	D	0
D	C	B	1

↳ Step 3, State assignment

- Choose # of flip flops, (check 2 this case y_2, y_1)
- Choose state codes, (A:00, B:01, C:10, D:11)

↳ Step 4, State - Assigned Table (rewriting step 2)

PS $y_2 y_1$	NS		z
	$w=0$ $y_2 y_1$	$w=1$ $y_2 y_1$	
00	00	01	0
01	10	01	0
10	00	11	0
11	10	01	1

↳ Step 5, Synthesize circuit

- So, we boolean algebra, K map etc. to obtain logic expression

We need expressions for next state, output

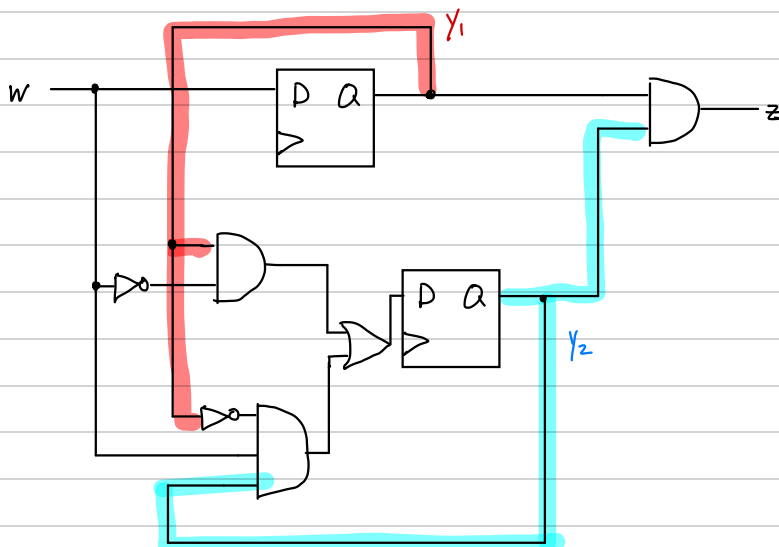
$$Y_1 = w$$

$w \backslash y_1$	00	01	11	10
0		1	1	
1				1

$$Y_2 = y_1 \bar{w} + y_2 \bar{y}_1 w$$

$$z = y_2 y_1$$

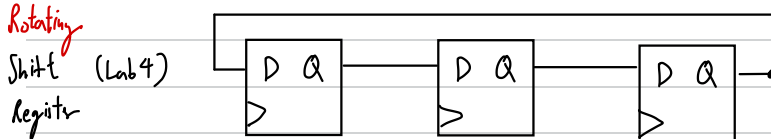
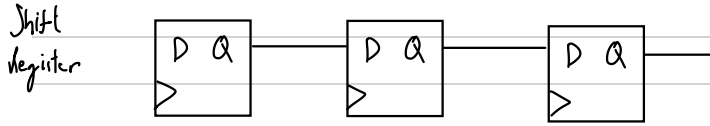
- Step 7b, draw circuit w/ flip flops



Special design choices

- A: 0001, B: 0010, C: 0100, D: 1000 ← One-hot encoding

- Letting y_2, y_1 shift digits to represent states ← Shift register



Verilog for FSM

```
module FSM (input logic w, clock, reset_n,
            output logic z);
```

```
    typedef enum logic [1:0] (A, B, C, D) state_type;
```

```
    state_type ps, ns;
```

① //FS

```
    always_ff @(posedge clock,
                negedge reset_n)
```

```
        if (!reset_n)
```

```
            ps <= A;
```

```
        else
```

```
            ps <= ns;
```

② //state table

Follow this

```
    case (ps)
```

```
        A: if (w) ns = B;
```

```
        else ns = A;
```

```
        B: if (w) ns = B;
```

```
        else ns = C;
```

```
        C: if (w) ns = D;
```

```
        else ns = A;
```

```
        D: if (w) ns = B;
```

```
        else ns = C;
```

```
    endcase
```

③ //out put

```
    assign z = (ps == D);
```

```
endmodule
```

PS (Present state)	Next state		Output z
	w = 0	w = 1	
A	A	B	0
B	C	B	0
C	A	D	0
D	C	B	1

Lab Misc

.do files

`vlib work` # creates design library of name "work", where verilog compiles
`vlog filename.sv` # compiles file ex. `vlog mux.sv`
`vsim modulename` # starts simulating the top level module ex. `vsim mux`

`log -r {/*}` # logs all objects
`add wave {/*}` # adds all objects of the top level module to the waveform

`force <object-name> <value>` # sets input <object-name> to value ex. `force {SW[0]} 0`
`run <time-steps> [<time-units>]` # advances simulation ex. `run 10ns`

Periodic signals

`force <object-name> <value-1> <time-1>, <value-2> <time-2> -r <time-3>`
ex. `force {clk} 0 0ns, 1 {5ns} -r 10ns`
1. sets clk to 0 after 0ns
2. sets clk to 1 after 5ns
3. Repeat cycle after 10ns



- Either force a clock to run
or force every test case

Verilog Syntax

↳ Reduction Operations: Multiple gates working on a single bus
ex. If logic `a[3:0]`, then
`&a = a[0] & a[1] & a[2] & a[3]`

↳ Right-shift:
`a >> b`, shifts `a` right by `b` bits

ex. `1001 >> 10 = 0010`

↳ Concatenation: Quite literally to concatenate the bits

↳ Left-shift:

ex. `y = {a[2:1], b[3:0]}` = `a[2] a[1] b[3] b[2] b[1] b[0]`
`y[5] y[4] y[3] y[2] y[1] y[0]`

`a << b`, shifts `a` left by `b` bits

ex. `1001 << 01 = 0010`

↳ Most / Least significant bits
leftmost bit rightmost bit

↳ Parameters, treated as constants in the code

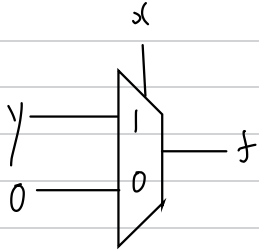
ex. module adder

#(parameter WIDTH = 8)
input logic [WIDTH-1:0] a, $\longrightarrow$ declares `a` as 8 bit bus.

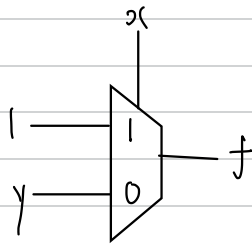
↳ `c log2( )`, log₂ function

Creating Gates with mux

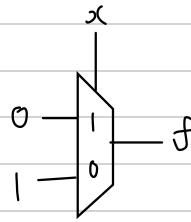
AND: xy



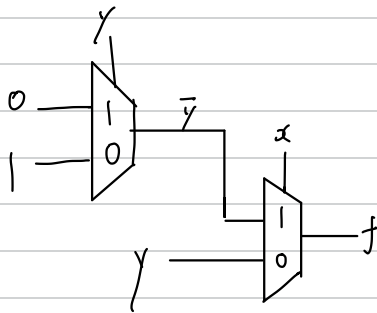
OR: $x+y$



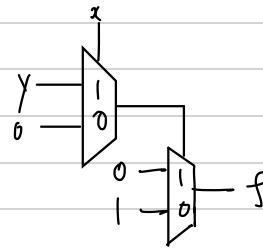
NOT: $\bar{x}$



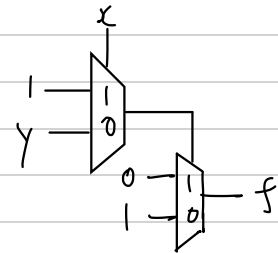
XOR: $\bar{x}y + x\bar{y}$



NAND: $\overline{xy}$



NOR: $\overline{x+y}$



Lecture 10

2's complement

↳ Method to represent negative number

↳ Most significant bit (MSB) is the sign bit

↳ 1 for negative #, 0 for positive #

↳ Using 2's complement reduces maximum value represented by half $\rightarrow$ from -2^{n-1} to $2^{n-1}-1$

↳ How to obtain -ve #:

Given k with n bits, $-k$ is $2^n - k$

ex. $k = 1000$, $n = 5$ bits

a) $2^5 - 2^5 = 32$

b) $32 - 8 = 24$

c) $-k = 11000$

★ Shortcut

a.) Complement each bit

ex. $0b0010 \rightarrow 0b1101$

b.) Add 1

ex. $0b1101 + 1 = 0b1110 = -2$

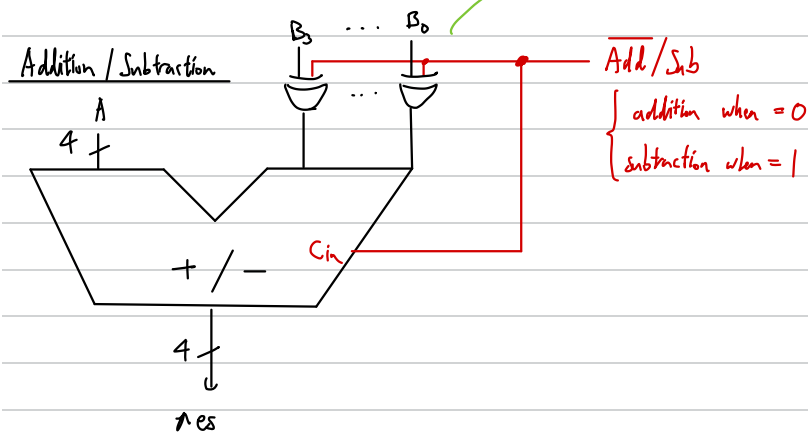
↳ Same number, more bits $\rightarrow$ Sign extension

ex. $n10010 \dots \rightarrow nnnn10010 \dots$

ex. $0b1011 \rightarrow 0b1111011$

$0b001 \rightarrow 00000001$

★ Shortcut to obtain -ve



Overflow

↳ When adding two same sign bits, arithmetic overflow occurred.

Overflow cannot occur if number has opposite signs

Overflow occurs when S_3 and C_{out} have different values

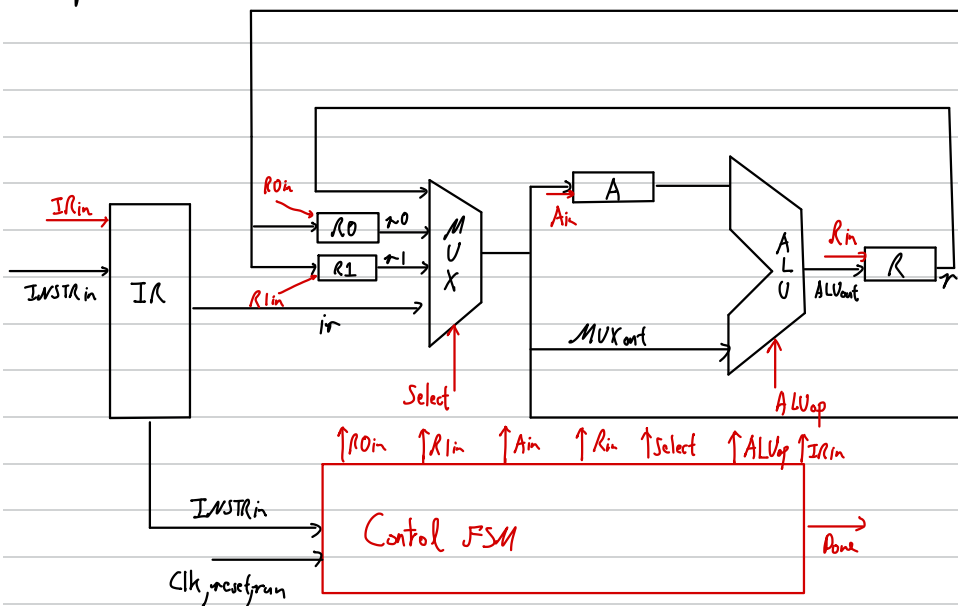
↳ Overflow = $S_3 \overline{C_{out}} + \overline{S_3} C_{out} = S_3 \oplus C_{out}$

Lecture 11

Simple Processor (Lab 5)

- ↳ A logic circuit
- ↳ Contains combinational, sequential logic, controlled by FSM
- ↳ Includes a set of n-bit general purpose registers
 - ↳ Holds values used in computations
- ↳ RISC-V has 32 32-bit registers, Simple processor has 2 16-bit registers
- ↳ Has ALU
 - ↳ Simple one does add, subtract, multiply
 - ↳ Complex one does more operations: AND, OR, XOR, shift, rotate etc.
- ↳ External interface
 - ↳ w/ memory, I/O devices (ex. keyboards), network, displays, hard drive
 - ↳ Address: who to communicate
 - ↳ Data: what to communicate
 - ↳ Control: read/write

Simple Processor



Data Path
Control Path

IR: Instruction register, holds instruction to execute

RO, R1: 16-bit general purpose registers, hold input to computation, result of computation

A: First (temporary storage) input to ALU

R: hold result from ALU prior storing result in RO or R1

Control FSM: Determines value of control signals depending on which step of current instruction

Instruction operands (where does the data the instruction compute on come from)

- ① Registers → 16-bit value stored in registers
- ② Immediate → Data value in the instruction

Instructions

Sign extension:

rX, rY

$rX \leftarrow rY$

$rX, \#D$

$rX \leftarrow \text{sign_extend}(D)$

$\{4\{INSTIN[11]\}\}, INSTIN[11:0]\};$

$\text{add } rX, rY$

$rX \leftarrow rX + rY$

$\text{add } rX, \#D$

$rX \leftarrow rX + \text{sign_extend}(D)$

etc...

OPCODE

II M X DDDD DDDD DDDD

II: Specifies instruction carried out

X: Specifies destination for register rX

$X=0, rX=r0$

M: Choose source data from register or immediate value

$X=1, rX=r1$

$M=0 \rightarrow DDDD, DDDD, DDDD = 0000, 0000, 000Y \rightarrow Y=0, rY=r0$

$M=1 \rightarrow DDDD, DDDD, DDDD = \#D$

$Y=1, rY=r1$

Control signals

	C0	C1	C2	C3
mv	IRin=1	Set ROin / Rin based on X Set Select based on M, Y Done = 1		
add	IRin=1		Rin = 1	Select = 00
		Set select based on X Set Rin = 1	Set select based on M, Y	ROin or Rin based on X
sub	IRin=1		ALUop = 00, 01, 10 (any 1)	Done = 1
mul	IRin=1			

Verilog

Control path (FSM)

① Flip Flips

a) Go to next state at clock b) Back to first state at reset

② State table

a) Check for done at C1 (done mv)

③ Set control signals, such as ROin / Rin, select, ALUOP etc.

Data path

① Choose item at mux

② Carry out math at ALU

③ Store everything in registers.

Assembly

↳ ADD R1, R0 is an assembly level instruction for a simple processor

↳ Instruction set architecture (ISA)

↳ Human readable form (not machine code)

↳ Compiler: C, Java $\xrightarrow{\text{translate}}$ Assembly

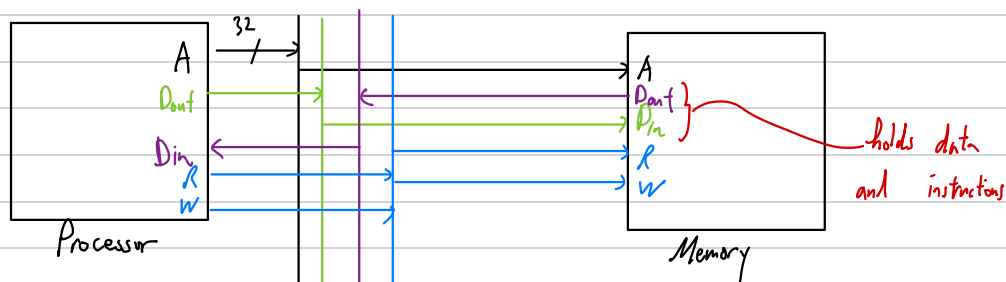
↳ Assembly language instruction specifies:

- Operation

- Operands

- Value in registers
- Constants (immediates)
- Memory

Computer organization



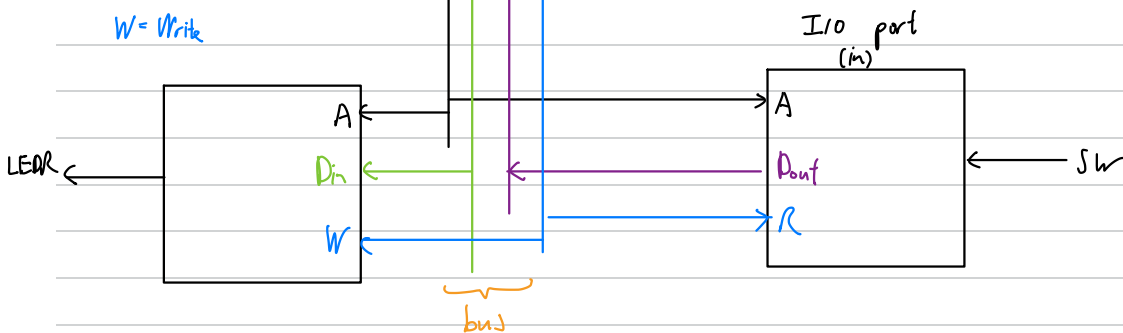
A: Address

Dout = Data Out

Din = Data In

R = Read

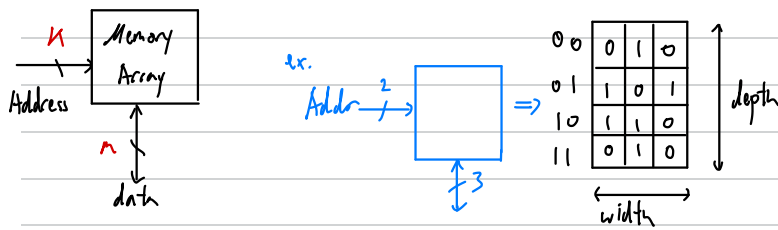
W = Write



↳ Memory & I/O ports assigned to a range of addresses called memory map (memory mapped I/O)
Only one device responds to unique address from processor

Memory Architecture

↳ Digital circuits (including processors) require memories to store data and instructions

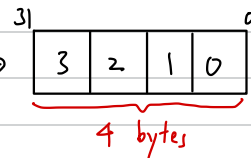


Like a 2D array (hold data) (k, n)

↳ Detailed view

Addr	Data
0	word 0
4	word 1
8	word 2
	⋮
	word $n-1$

1 word = 4 bytes = 32 bits
1 byte = 8 bit



4 bytes
byte addressable memory

↳ k address bits = 2^k addresses → each 4 addresses has 4 bytes
↳ ∴ 2^k bytes / k address bits
 2^{k-2} words / k address bits

↳ Address $A_7 \dots A_2 A_1 A_0$
pick word (row) pick byte (column)

Lecture 12

↳ Different assembly languages:

CISC, complex instruction set computer, x86

RISC, reduced instruction set computer, ARM, RISC-V

↳ RISC-V 32I → 32 registers, each 32 bits

Instruction types

⊗ Add: add $s0, s1, s2$
Source operands: $s1, s2$
Destination operand: $s0$

i.e. $s0 = s1 + s2$

Combined: add $t0, s1, s2$

sub $s0, t0, s3$

i.e. $s0 = s1 + s2 - s3$

⊗ Sub: sub $s0, s1, s2$

i.e. $s0 = s1 - s2$

↳ Principle: Regularity supports simplicity

Register set

↳ $x0 - x31$, typically call with special names

- ① zero: $x0 \rightarrow$ always zero
- ② $s0 - s31$, $t0 - t6$: general purpose, store variable
- ③ $ra, a0 - a7$: special uses for function calls
- ④ sp, gp, tp : tl

Constants

↳ Immediately available from instruction rather than a register/memory

✗ `addi s0, s0, 4`

↳ $s0 = s0 + 4$

No `subi`

↳ `s1, s0, -12`

$s1 = s0 + (-12)$

$= s0 - 12$

Initializing w/ immediates: `addi s0, zero, 67` $\rightarrow s0 = 67$

Sign extension

↳ Immediates are 12-bit, 2's complement numbers

Sign extend makes it 32-bit

↳ If immediate > 12 bits, use `lui` $\rightarrow$ load 20 bit immediate into most-significant 20 bits in register, fills the rest w/ 0

✗ `lui s2, 0xABCD E` $\rightarrow s2 = 0xABCD E000$

`addi s2, s2, 0x123` $\rightarrow s2 = 0xABCD E123$

$\rightarrow$ Pseudo instr. `li`:

`li s2, 0xABCD E123`

(Pseudo instrs are not actual part of the language,
commonly used/convenient for programmers)

Memory instructions

✗ `load word, lw s1, "offset" ("base")` (Loads data word from mem into register)

↳ addr $\rightarrow$ evenly divisible by 4

e.g. `lw s1, 8(zero)` (say $a = \text{mem}[2]$)

$\boxed{2 \times 4}$
index word size

$\nwarrow$ word 2

✗ `store word, sw s2, "offset" ("base")` (Writes data word from a register into mem)

e.g. `sw s2, 12(zero)` (say $s2$ has $0x42$)

Then $\text{mem}[3]$ has $0x42$ now

Sample code

data

LIST: word 1, 2, 3, 4

randomly assigned #s

word-sized memory locations

global _start

text

start:

la s1, LIST

lw s2, 0(s1)

lw s3, 4(s1)

add s2, s2, s3

lw s3, 8(s1)

add s2, s2, s3

lw s3, 12(s1)

add s2, s2, s3

END: ebreak

Assembler directives:

↳ Guide assembler in allocating, initializing global variables, defining const., differentiating b/w code & data (Not part of the code)

List:

↳ A label to allow you to refer to stuff in the code

↳ Changes into address

↳ No need to figure out actual addresses

-start:

Like main in C

-ebreak:

Tells processor to stop executing

★ ↳ load address, la s1, LIST (pseudo instr to load address, "put address of LIST in s1")

More instructions

↳ Logic instrs. (bit-wise operation)

and s1, s1, s2 0000 0000

or s1, s1, s2 1111 1111

xor s1, s1, s2 1111 1111

not s1, s1 1010 1010

← pseudo instr, xori s1, s1, -1

say s1 = 0101 0101...

s2 = 1010 1010...

↳ Shift instrs

shift left logical, sll (fill least sig bits w/ 0s)

shift right logical, srl (fill most sig bits w/ 0s)

shift right arithmetic, sra (fill most sig bits w/ sign bit)

ex. s5 = 1001 1011 ...

slli s0, s5, 5 → s0 = 11011...000

srls s0, s5, 5 → s0 = 0001001 1011...

sras s0, s5, 5 → s0 = 111 1001 1011...

Multiply by $2^n \rightarrow$ shift left by n
Divide by $2^n \rightarrow$ arithmetic shift by n

32-bit, 1500

↳ Accessing bytes, half-word of memory

* load byte, lb
load byte unsigned, lbu

↳ lb sign extends upper 24 bits
lbu fills with 0s

load half word, lh (2 bytes)

load half word unsigned, lhu

store byte, sb, stores least sig. byte to mem, changes one byte only
store half word, sh, same with above but 2 bytes

example.

lbu s1, 3(s4) can be any number.
lb s2, 5(s4)
lh s3, 2(s4) half words divisible by 2
lhu s5, 0(s4)
sb s2, 0(s4)

Program Flow

↳ Instr stored in memory, 4 bytes long
↳ Consecutive instr. addresses increase by 4

↳ Program Counter (PC): holds memory of current instr, increments by 4 after each instr. completes

- ① Read instr from mem
- ② Decode instr
- ③ Execute instr
- ④ Increment PC by 4

Branching

↳ If/else is high-level language.

↳ Modify program flow, so processor can execute instructions not in sequential order.

o Conditional branch

↳ Perform a test $\rightarrow$ branch to new instr if test true

↳ All compare 2 source registers

beq: branch if equal

bne: branch if not equal

signed: blt: branch less than, bge: branch greater than or equal

unsigned: bltu, bgeu

pseudos: beqz, bnez, blez, bgtz, ble, bgt, bleu, bgtu
↳ 2 for zero

o Unconditional branch instr. (jump-always)
 j LABEL, jal LABEL (jump-like), jr (jump register)

Example uses

Loop1: addi s8, s8, -1 increment s8 by -1,
 bnez s8, Loop1 go back if s8 ≠ 0
 # instr.

Loop2: addi s8, s8, 1 increment s8 by 1
 bgeq s8, s9, Loop2 go back if s8 ≥ s9

If, then, else

Use if (s8 > s9) as example,

bgt s8, s9, THEN path 1 if s8 > s9
 ELSE: # ELSE code path 2 if s8 <= s9
 j AFTER ← skips then
 THEN: # THEN CODE
 AFTER: # AFTER CODE

For loop

use for (s8=1; s8<5; s8++) as example

LOOP1: bgt s8, 5, AFTER
 # CODE
 j LOOP1
 AFTER: # CODE

While loop

use while (s8 > 8) as example

LOOP2: bgt s8, 8, AFTER2
 # CODE
 addi s8, s8, 1
 j LOOP2
 AFTER2: # CODE

Lab 06

{ slli s2, s3, 1 left shift →
 { and s3, s3, s2 bit-wise and

ex 101111001
 011110010
 and → 001110000 ...

each shift then "and"

Lecture 13

↳ Subroutine → make code modular, easy to reuse
 input argument, return values

↳ Branching to subroutines

- need to return to instructions after calling subroutine

- use jal LABEL

↳ jal saves PC of instr after it into ra (return address register)

- to return from a subroutine

jr m
jump register

Example function

C code

```
int main() {  
    simple();  
    #random instr.  
}
```

```
void simple() {  
    return;  
}
```

Code

jal SIMPLE
#random instr.

new addr

0x00000209

0x00000308

SIMPLE: jr ra. 0x000051C

After this, $ra = 0x00000308$,
 $PC = 0x000051C$

After this, $PC = 0x00000308$

Arguments, return value

↳ Conventions

caller → function calling subroutine

callee → subroutine being called

① caller & callee must agree where to put arguments, return values

② callee must not interfere w/ behaviour of caller

caller must not clobber/overwrite any register or memory used by callee.

↳ Calling conv. for arguments

use registers a0 to a7

"function arguments left to right in a0 to a7 as needed"

↳ Calling conv. for return value

return always goes in a0

Example

```
int main() {  
    return add6(11, 22, 33, 44, 55, 66);  
}  
  
int add6(int a, int b, int c, int d, int e, int f) {  
    return a + b + c + d + e + f;  
}
```

Code:

- start: # 1. put arguments into registers

```
addi r0, zero, 11
```

addi r1, zero, 22

addi r2, zero, 33

addi r2, zero, 44

```
addi r4, zero, 55
```

addi r5, zero, 66

jul add 6

END: ebreak

add 5:

add sl, a0, r1 #sl = 33

addsz, a2, a5 # sz=77

add $s_3, a_4, a_5 \neq s_3 = |2|$

add s_1, s_1, s_2

add s1, s1, s3

add a0, sl, zero # addn value $\leftarrow$ returned in a0

in na return to END

Stack

↳ LIFO structure: Last in first out

Grow towards lower addr.

... FFF8

0x3FFF FFFC

push data onto stack

↓ pop data ft

Sp

(sp)

Stack pointer, points to top of the stack

Example

Before

After

JFF4		... FFF9	FFFF	← sp
... FF8		.. FFF0	1234 5678	
0 * JFFFJFFC	4BCD 0000 ← sp	... FFFC	ABCD 0000	

Adding words $\rightarrow$ decrease sp
"sp decrease by 8 to add 2 words"

Saving / restoring registers

↳ Functions should not tamper registers of the caller function

↳ Preserved registers: Must contain same values at beginning and end of called function,
caller expects them to be the same after call
s0-s11, ra, sp

↳ Non preserved registers: do not need to be preserved, scratch or temp.
t0-t6

Fixing example

Code:

start: # put arguments into registers

addi s1, zero, 77
addi s2, zero, -1
addi s3, zero, 0 } now initialized s1, s2, s3
must be preserved

addi a0, zero, 11

addi a1, zero, 22

addi a3, zero, 33

addi a3, zero, 44

addi a4, zero, 55

addi a5, zero, 66

jrl add6

sub s4, s1, s2

sub s5, s3, s1

END: ebreak
add6:

add s1, a0, a1 # s1 = 33

add s2, a2, a3 # s2 = 77

add s3, a4, a5 # s3 = 121

add s1, s1, s2

add s1, s1, s3

add a0, s1, zero # return value ← return is a0

jra ra return to END

should not be tampered.

New add6:

add6: addi sp, sp, -12 # allocate 3 word loc. on stack

sw s1, 8(sp) ②

sw s2, 4(sp) ③

sw s3, 0(sp) ④

add s1, a0, a1 # s1 = 33 ⑤ Usual code

add s2, a2, a3 # s2 = 77

add s3, a4, a5 # s3 = 121

add s1, s1, s2

add s1, s1, s3

add a0, s1, zero # return value ← return is a0

lw s3, 0(sp) ⑥ return values (Last in first out)

lw s2, 4(sp)

lw s1, 8(sp)

addi sp, sp, 12 ⑦ return pointer

Stack diagram:

		① new sp address from step 1
FF0	0 ④	← sp
FF4	-1 ③	
FF8	77 ②	
FFC	???	← sp ← sp ⑦ return sp. We don't know/care what's here

Nested subroutines

- ↳ subroutines that call other subroutines
- ↳ remember multiple return addresses
- ↳ Only 1 ra exists → we stack

add6 example (again)

C code:

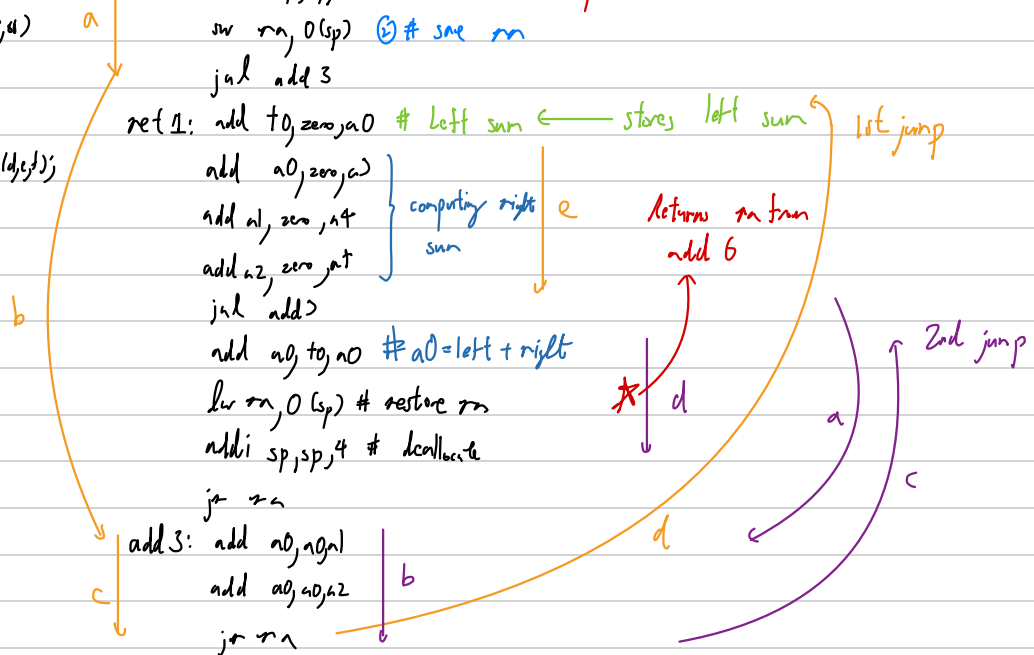
```
int main() {
    return add6(1, 2, 3, 4, 5, 6);
}

int add6(int a, ...) {
    return add3(a, b, c) + add3(d, e, f);
}

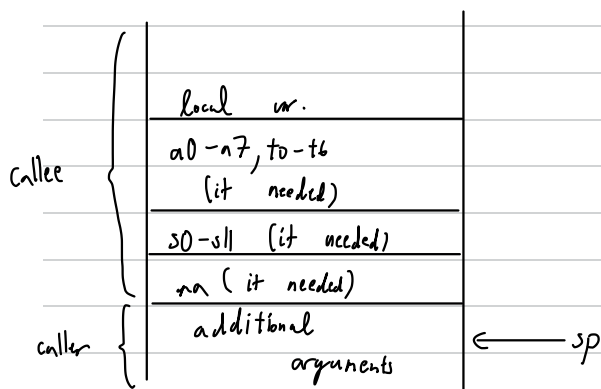
int add3(int x, y, z) {
    return x + y + z;
}
```

Assembly:

```
add6: add sp, sp, -4 ① # allocate stack space
      sw ra, 0(sp) ② # save ra
      jal add3
ret4: add t0, zero, a0 # left sum ← stores left sum
      add a0, zero, a1
      add a1, zero, a2 } computing right
                       } sum
      add a2, zero, a3
      jal add3
      add a0, t0, a0 # a0 = left + right
      lw ra, 0(sp) # restore ra
      addi sp, sp, 4 # deallocate
      jr ra
add3: add a0, a0, a1
      add a0, a0, a2
      jr ra
```



Stack passing arguments, local variables



Callee save: s0-s11, ra, sp ← restore before return

Caller save: t0-t6, a0-a7 - it needed after call to subroutine

Sum example

C code:	Assembly:
int FINDSUM(int N) {	.data
int sum = 0;	N: word 5
while (N != 0) {	.text
sum = sum + N;	.global _start
N = N - 1;	_start: la s4, N
}	lw a0, 0(s4) # a0 = 5
return sum;	jal FINDSUM # r4 = END
}	END: ebreak
	FINDSUM: addi t0, zero, 0 # holds sum
	LOOP: beqz a0, ENDLOOP
	add t0, t0, a0 # sum = sum + N
	addi a0, a0, -1 # N = N - 1
	j LOOP
	END LOOP: addi a0, t0, 0 # return sum in a0
	jr ra

Recursion

C code:	Assembly:
int FINDSUM(int N) {	.text
if (N == 0)	.global _start
return 0;	_start: la s4, N
else	lw a0, 0(s4) # a0 = N
return (N + FINDSUM(N-1));	jal FINDSUM
}	END: ebreak
	FINDSUM: bnez a0, RECURSE
	jr ra # if N=0, return 0
	RECURSE: addi sp, sp, -8
	sw a0, 4(sp)
	sw ra, 0(sp)
	addi a0, a0, -1
	jal FINDSUM # recursive call overwrites ra and a0
	EEE: lw t0, 4(sp) # get my N back
	add a0, a0, t0 # a0 = N + FINDSUM(N-1)
	lw ra, 0(sp) # restore ra
	add sp, sp, 8
	jr ra

① this jr ra brings you to EEE
 ② keep on repeating
 ③ N + FINDSUM(N-1)
 Return to
 { EEE if not the end
 { END else.

Lab 07

Pseudo Code 1:

```
cur_max = 0
listlen = list[0]
while listlen != -1:
    consec-1 = ARES(listlen)
    if consec-1 > cur_max:
        cur_max = consec-1
    listlen = list + 1
```

Pseudo Code 2:

```
def bubble_sort:
    length = list[0]
    for i in range(length):
        for j in range(0, length-i-1):
            SWAP(list[j])
```

SWAP:

```
swapped = 0
if list[j] > list[j+1]:
    list[j+1] = list[j]
    list[j] = list[j+1]
    swapped = 1
return swapped
```

Lecture 14

I/O

↳ Connecting device to CPU

↳ Memory is a device and CPU communicates with it through:
load and store instructions:

Operands: ^(lw) address and ^(sw) data

↳ Memory responds to certain addresses (usually not all)

↳ CPU can talk with other devices (using lw, sw)

↳ Devices will not respond to certain addresses

Memory mapped I/O

↳ Device sits on memory bus

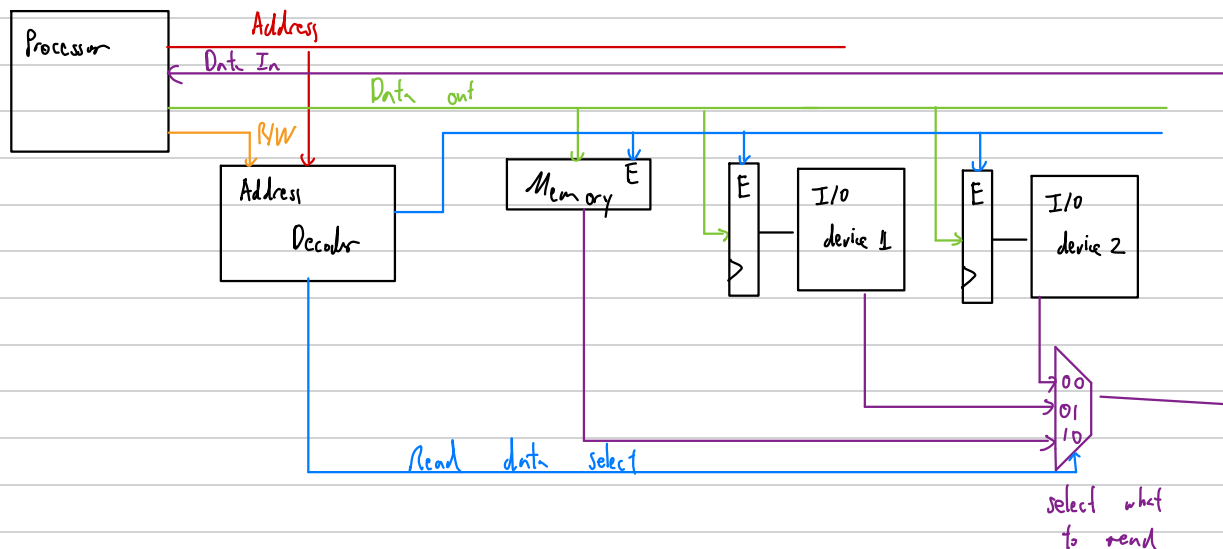
writes for certain addresses

respond like memory for those addresses

"read" memory ignores those addresses

Memory map

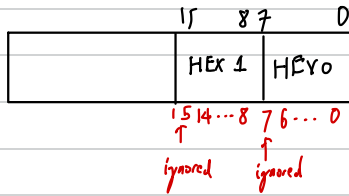
Map of which devices respond to which addresses



HEX example

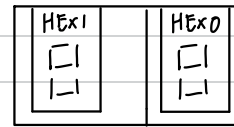
Addr
0x5F20 0020

Mem structure



Actual display

5 | 6 | 1 |
4 | 1 | 2 |
5



This file, 1 → ON

Assembly

.data

0011
1111

hex_codes: .byte 0x3f, 0x06, 0x5b, 0x4f, 0x66

.byte 0x6d, 0x7d, 0x07, 0x7f, 0x67

.text

.global _start

_start:

li s2, 0x4200020 # right hex 0x420 0020, left hex 0x420 0021

initialize right digit

★ mv s0, zero # pseudo code, addi s0, zero 0

mv s3, zero

mv s4, zero left digit

li s5, 0x00000000

mv a0, s4 # need hex decoder to convert number

jal HEX_DECODE to segments

mv s3, a0

slli s3, s3

Polling

↳ Iteratively check to see if a device is ready

Timer example

	31	...	16	15	...	3	2	1	0
0xFF202000							RUN		TO
0xFF202004							STOP	START	COM
0xFF202008							period		ITO
... 200C							Counter start val (low)		
... 2010							Counter start val (high)		
... 2014							Counter snapshot (low)		
							Counter snapshot (high)		

RUN: 1 if timer is running

TO: 1 if time out has occurred

START: 1 to start

COM: 1 to keep timer running after timeout

Goal:

↳ To blink LEDx0 every 1 sec

↳ Located at 0xFF200000

31	9	0
		LEDx5

- eqn PERIOD, 100,000,000
- eqn TIME, 0xFF202000
- eqn LEDx, 0xFF200000
- global - start

• eqn - assembly directive - like # define

- start

li s0, TIME # s0 = base address of timer

li s1, PERIOD # s1 = timer period value

li s4, LEDx # s4 = base address of LEDx

mv s7, zero

li s7, 0x0000FFFF PERIOD separated into lower, upper 16 bits

slli s2, s1, 16 # s2 has upper 16 bits

and s1, s1, s7 # s1 has lower 16 bits

I/O

sw s1, 8(s0) # store period to corresponding loc. in mem map

sw s2, 12(s0)

addi s3, zero, 6 # f = 0b110

sw s3, 4(s0) # sw → starts

POLL: lw s0, 0(s0) # poll timer

andi s6, s0, 1 # bit zero of s0 will have t/o bit

breq s6, POLL # if t/o bit is zero, don't start

xori s7, s7, 1 # xor to turn LED on/off

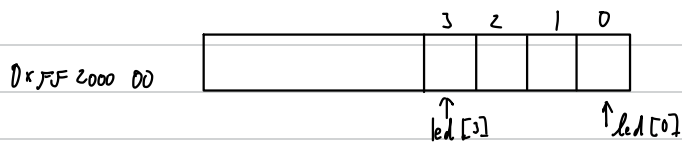
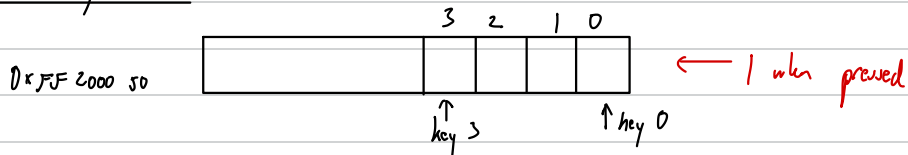
sw s7, 0(s4) # I/O for LEDx - store new val

sw zero, 0(s0) # zero will reset timer

j POLL

Poll makes resources

Buttons, LEDs



• text

- global_start

_start:

li s0, 0x5F20 0070 # s0 holds address of key

li sl, 0x FF20 0000 # sl holds address of LEDs

POLL: h_w sz, $O(sz)$ if poll or key pres

beg₃ ¹², POLL # branch to POLL if no key pressed

WAIT: $\ln 53, 0 (10)$

bnez s3, WAIT # unit for key release

li s₁ 060001 # s₂ will be 1 it

bnc s2, s3, CHECK-1 # key 0 was pressed

li 54, 0 # value to be written to LEDs

UPDATE-LED

CHECK 1: $l: s3, 0b\ 00\ 10$

```
bne s2, s3, CHECK_2
```

$li_{s4,1}$ # value to be written to LED₃

```

; UPDATE-LED

```

CHECK-2: 11 55, 06 0100

```
bne r2, s3, CHECK-3
```

li 5,3 # value to be written to LED)

```

; UPDATE_LED

```

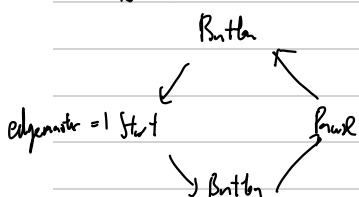
CHECK-3: Is s_4 a value to be written to LED?

UPDATE - LED:

sw s4, 0(s1) # light up LEDs

j POLL

Lab 08



Interrupts

↳ Polled I/O can be inefficient, prevent processor from doing other work
↳ Instead execute code and let device interrupt processor

↳ Basics

- 1. CPU configures itself for interrupts
e.g. device, memory, etc.
- 2. CPU does other work
- 3. CPU gets interrupted - can be any time!
 - ↳ CPU saves state, e.g., PC
 - ↳ CPU jumps to interrupt handler
 - ↳ CPU resumes what it was doing

	Polling	Interrupt
When?	Explicit polling loop in code	Anytime (once setup)
Diff.	Easier	Complex
Efficiency	Good if small wait is expected	Good if medium to long waits

Example events by interrupts

External device: UART, USB

Operating sys (os): timer, disk I/O, keyboard I/O

Debugging: break points

Program errors:

- ↳ Divide by 0
- ↳ misaligned memory access
- ↳ memory protection violation (seg fault)

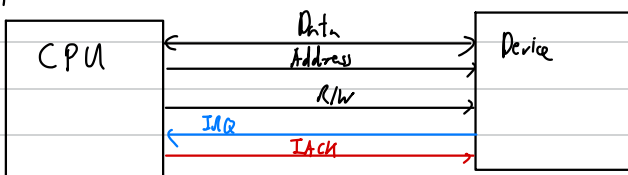
Exception, interrupt

Exception → arise within CPU, e.g. program error

Interrupt → Outside processor

Here interrupt describes both things

Interrupt Hardware

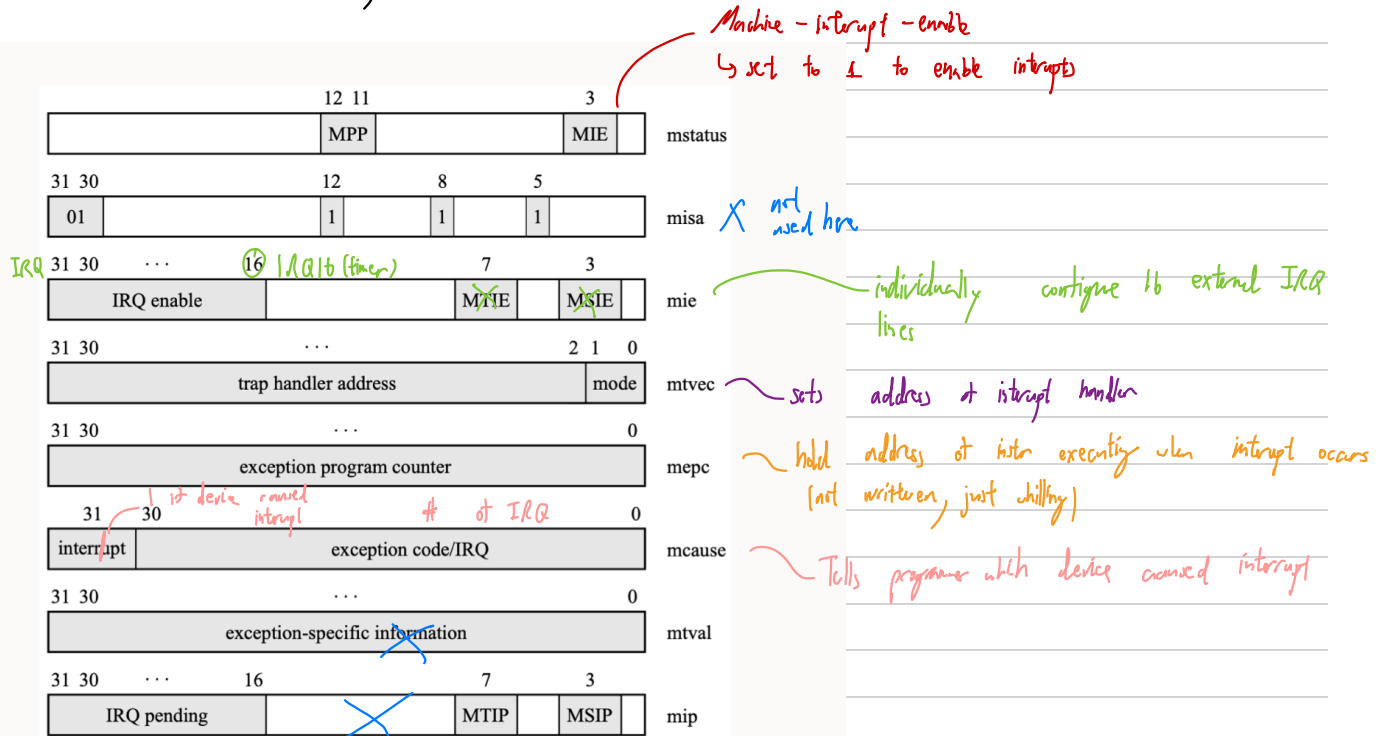


IRQ: interrupt request → device asserts to interrupt CPU

IACK: interrupt acknowledge → CPU asserts to acknowledge device's interrupt

Control and status registers (CSRs)

- ↳ Special registers to monitor system state
- ↳ only be read/written via special instructions.
- ↳ Each CSR has a special name, unique function. We focus on a few,



System instructions

mret - return from handling interrupt,
copies mepc value to program counter

csrr rd, csr # copy value of csr into

rd

→ any general purpose register e.g. s0, s1, ...

csw csr, rd # copy value from rd into csr

What happens

- ① CPU saves PC of interrupted instr into mpc register
- ② CPU jumps to address of interrupt handler (value contin into mtrac)
- ③ CPU executes handler code & then returns to interrupted instr.
 * must copy mpc into PC

Interrupt steps

- ① Disable interrupt (set mie = 0 in mstatus)
- ② Configure device
 Steps are specific to device
- ③ set specific IRQ lines in mie register
 set mtrac
 set mie bit in mstatus

Timer example

	31 ... 16 15 ...	3	2	1	0
0xFF202000				AVR	TD
0xFF202004		STOP	START	COUNT	INT0
0xFF202008		period { Counter start val (low)			
... 200C		Counter start val (high)			
... 2010		Counter snapshot (low)			
... 2014		Counter snapshot (high)			

write 1 to enable timer to generate interrupts

Timer connected to IRQ16

Task: Increment a counter with every timer interrupt

.data

COUNT: .word 0

.text

.global _start

.equ TIMER, 0xFF202000

_start: li sp, 0x200000

① csw mstatus, zero # turn off interrupts while setting up

② jal CONFIG_TIMER

③ # enable interrupts, set address of interrupt handler

li t0, 0x10000 ← bit 16 # to enable IRQ16

csw mie, t0 # IRQ16 = 1

la t0, interrupt-handler

csw mtrac, t0 # set address of interrupt handler

li t0, 0b1000

csw mstatus, t0

END: j END

CONFIG_TIMER:

configure timer period (like in polling)

```

li t1, TIMER
li to, 0b11 # start cnt, interrupt
sw to, 4(t1) # timer starts running
jr ra
interrupt_handler:

```

```

    addi sp, sp, -8
    sw s0, 0(sp)
    sw s1, 4(sp)
    csrc s0, mcause # who caused the interrupt

```

check {

```

    li s1, 0x0000 0010
    bne s1, s0, action

```

counter increment {

```

    la s0, COUNT
    lw s1, 0(s0)
    addi s1, s1, 1 # increment counter
    sw s1, 0(s0)

```

clear interrupt {

```

    li s0, TIMER
    sw zero, 0(s0) # clear interrupt by writing zero to status register

```

```

return: lw s0, 0(sp)
        lw s1, 4(sp)
        addi sp, sp, 8
        mret

```

Push button keys

	31	30	...	4	3:0	
0xFF200050						KEY3-0
0xFF200058						Mask bits
0xFF20005C						Edge bits

Interrupt mask register
Edge capture reg.

CONFIG_KEYS:

```

{ li t0, 0b1111 # all 4 keys to interrupt if pressed.
  sw to, 8(t1) # setting up interrupts for all keys
  sw to, 12(t1) # write 1 to edge capture to clear each bit
  jr ra

```

setting up interrupt in _start → same as before
 keys = IRQ 18
 li to, 0x40006 # 1 at bit 18 for IRQ 18
 csrc aie, to

interrupt - handler

save regs to stack

li \$1, 0x FF200050

lw \$0, 12(\$1) # read edge capture reg

do something

li \$0, 0b1111

sw \$0, 12(\$1) # write 1s to clear edge capture bits

restore stack

ret

Back to Lecture 12

Machine code

↳ Translating assembly into machine code

↳ 32 bit instr.

↳ RISC has 4 instr formats

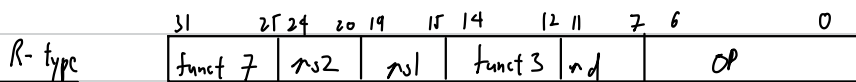
R type register

I type immediate

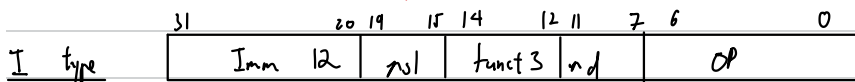
(no dest reg) S/B type store/branch

U-J type upper immediate/jump

DONT INCLUDE IN CHEAT SHEET



source reg 2 source reg 1



immediate (no source, funct 7) source reg 1

S/B, U-J Not covered.

Lecture 31

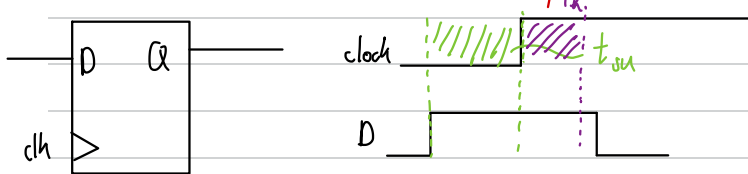
Timing analysis

↳ Logic **DOES NOT** operate with 0 delay

↳ Gates, wires have delay associated

determines clock frequency
determines whether or not circuit operates correctly

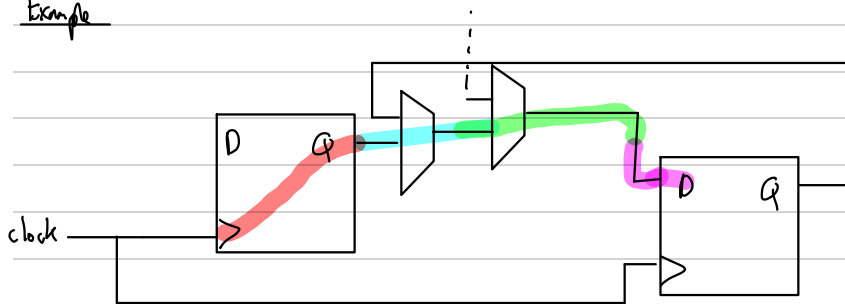
⊕ $t_{ca} \rightarrow$ "clock to Q" propagation delay



setup time: t_{su} - The data D must be stable during the setup time

hold time: t_h - The data must remain stable for the hold time

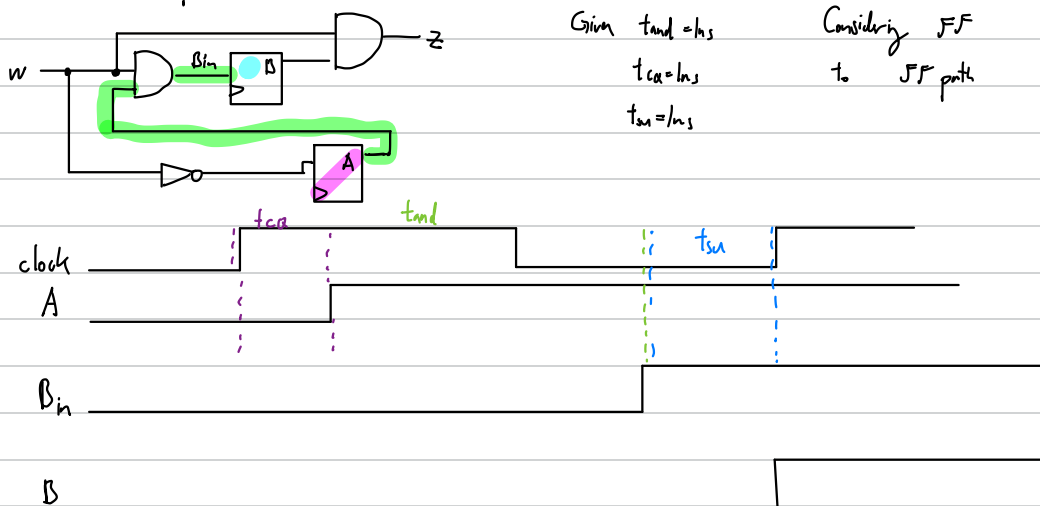
Example



$$T_{in} \geq t_{ca} + t_{mux} + t_{mux} + t_{su}$$

$$F_{max} = 1 / T_{in}$$

Another example



Given $t_{and} = 1ns$

$t_{ca} = 1ns$

$t_{su} = 1ns$

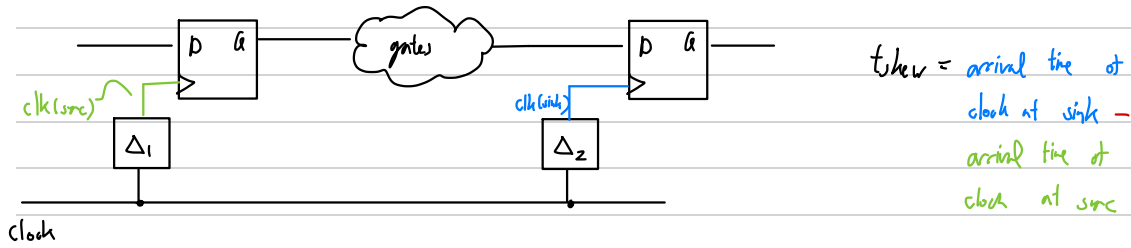
Considering FF
to FF path

$$T_{in} \geq t_{ca} + t_{and} + t_{su} = 1ns + 1ns + 1ns$$

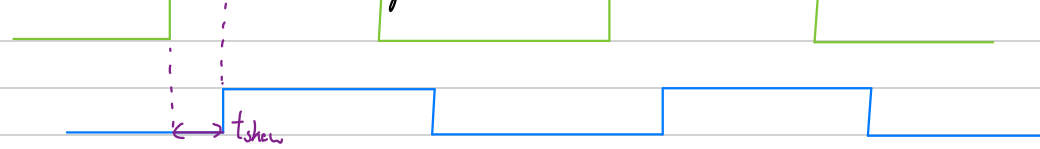
$$F_{max} = 33.3MHz$$

Clock skew

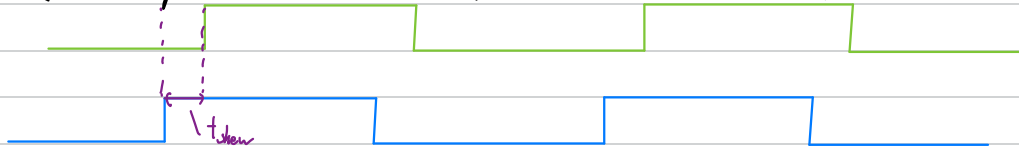
↳ Clock edge don't arrive at same time for all flip flops



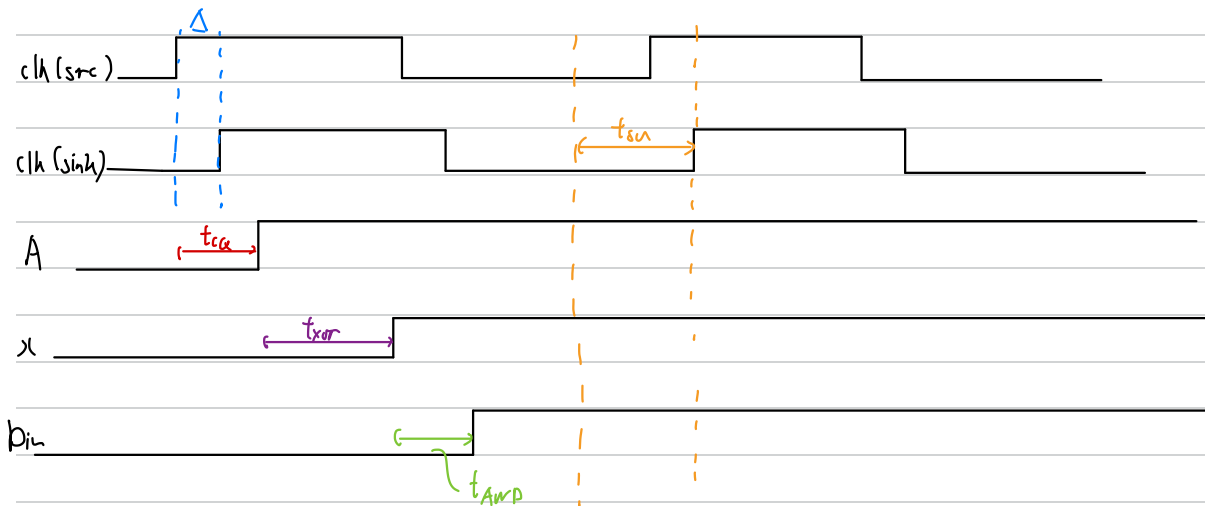
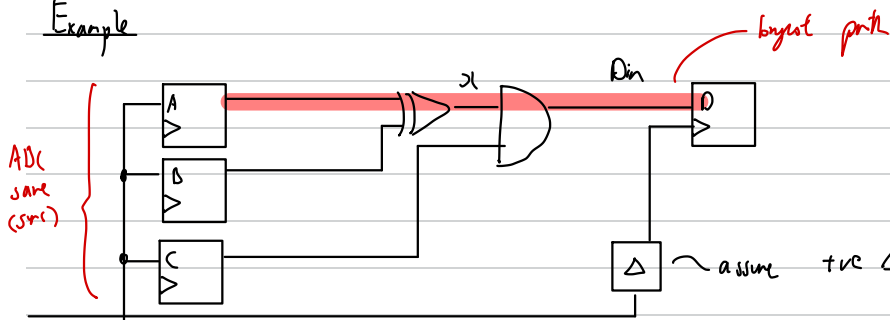
Case 1: Positive skew $\rightarrow$ gives more time $\rightarrow$ Higher F_{max}



Case 2: Negative skew $\rightarrow$ gives less time $\rightarrow$ Lower F_{max}



Example



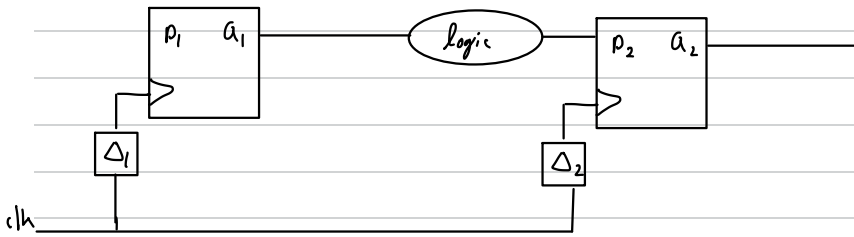
$$T_{in} + \Delta \geq t_{ca} + t_{xor} + t_{AND} + t_{sn}$$

$$T_{in} \geq t_{ca} + t_{xor} + t_{AND} + t_{sn} - \Delta$$

skew $\rightarrow$ beneficial here

Timing analysis steps

- ① Calculate clock, look for longest path b/w 2 FFs, includes setup time
- ② Calculate hold time violation
 - ↳ Shortest path b/w 2 FFs
 - ↳ Data could be racing the clock (setup time not used here)

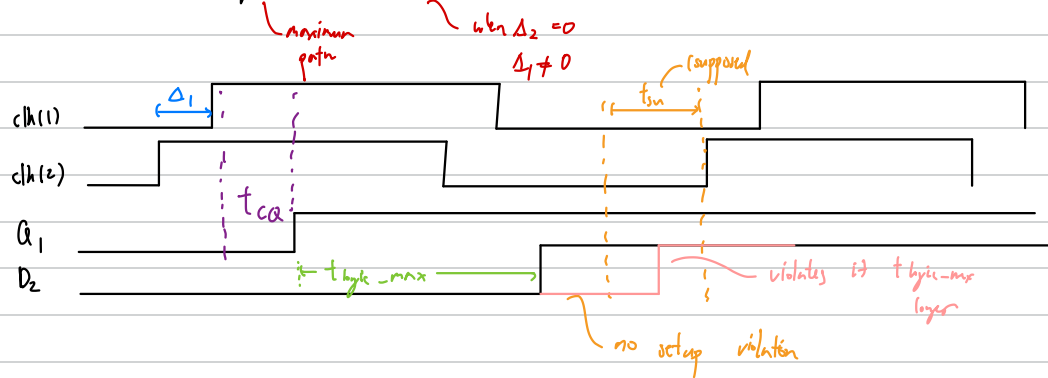


③ Setup time violation,

↳ logic too slow for correct value to arrive at input of FF on right D_2 before t_{su} , before clock edge.

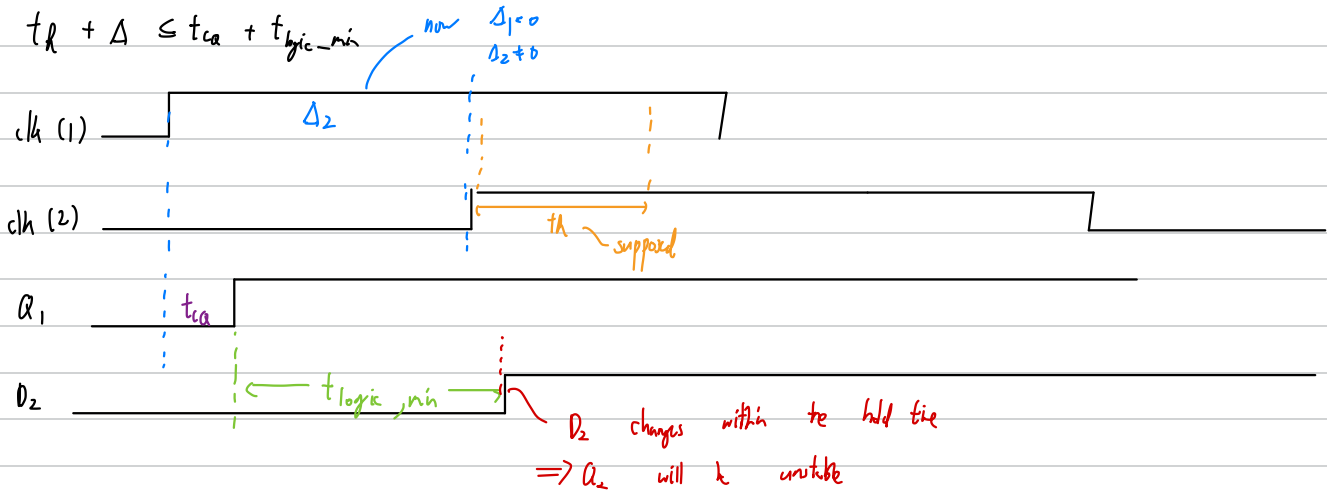
Setup time violation constraints

$$T_{in} \geq t_{ca} + t_{logic_max} + t_{su} + \Delta_1$$



Hold time violations constraints

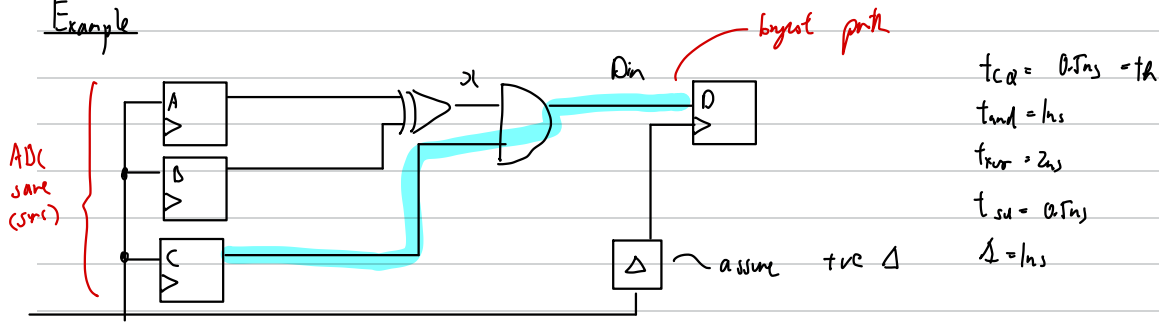
$$t_h + \Delta \leq t_{ca} + t_{logic_min}$$



If $t_{ca} + t_{logic_min} \leq t_h + \Delta \rightarrow$ hold time violation

Fixing hold time violations $\rightarrow$ insert additional logic, like $\rightarrow \neg \neg$

Example



Hold time violation:

$$t_h + \Delta \leq t_{ca} + t_{and}$$

$$0.5ns + \Delta \leq 0.5ns + 1ns$$

$$\Delta \leq 1ns \quad \leftarrow \text{exactly okay}$$

Set-up time violation:

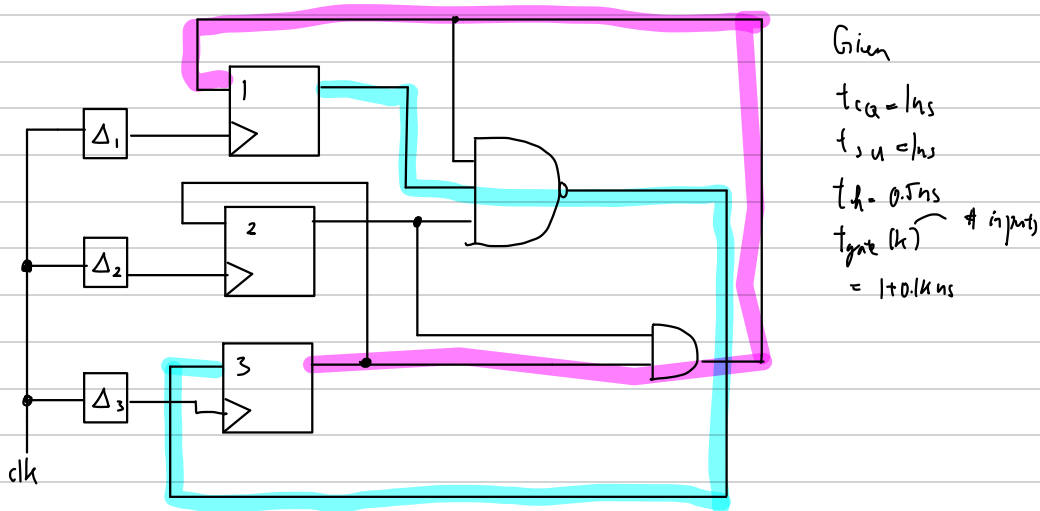
$$T_{in} + \Delta \geq t_{ca} + t_{or} + t_{and} + t_{su}$$

$$T_{in} \geq 0.5ns + 2ns + 1 + 0.5 - 1$$

$$\geq 3ns$$

if $\Delta = 1/2ns$, then hold time violation

Example 2



If $\Delta_1 = \Delta_2 = \Delta_3 = 0s$

Path 1 $\rightarrow$ 3

$$T_{in} \geq t_{ca} + t_{and} + t_{su} - 0$$

$$\geq 1 + 1.5 + 1 = 3.5ns$$

Path 3 $\rightarrow$ 1

$$T_{in} \geq t_{ca} + t_{and} + t_{su} - 0$$

$$\geq 1 + 1.2 + 1 = 3.2ns$$

$$T_{in} \geq 3.5$$

Calculate $\Delta_2 - \Delta_3$ that do not violate hold time $t_h + (\Delta_2 - \Delta_3) \leq t_{ca} + t_{gate}$

$$\Delta_2 - \Delta_3 \leq 1ns + 0.1ns$$

$$\Delta_2 - \Delta_3 \leq 0.5ns$$

